UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Dipartimento
di Fisica
e Astronomia
Galileo Galilei

No U-Turn Sampler

Adaptively Setting Path Lengths in Hamiltonian Monte Carlo

Pieripolli Leonardo, Pirazzo Tommaso,
Ponchio Mattia, Secco Benedetto

August 27, 2026

No U-Turn Sampler

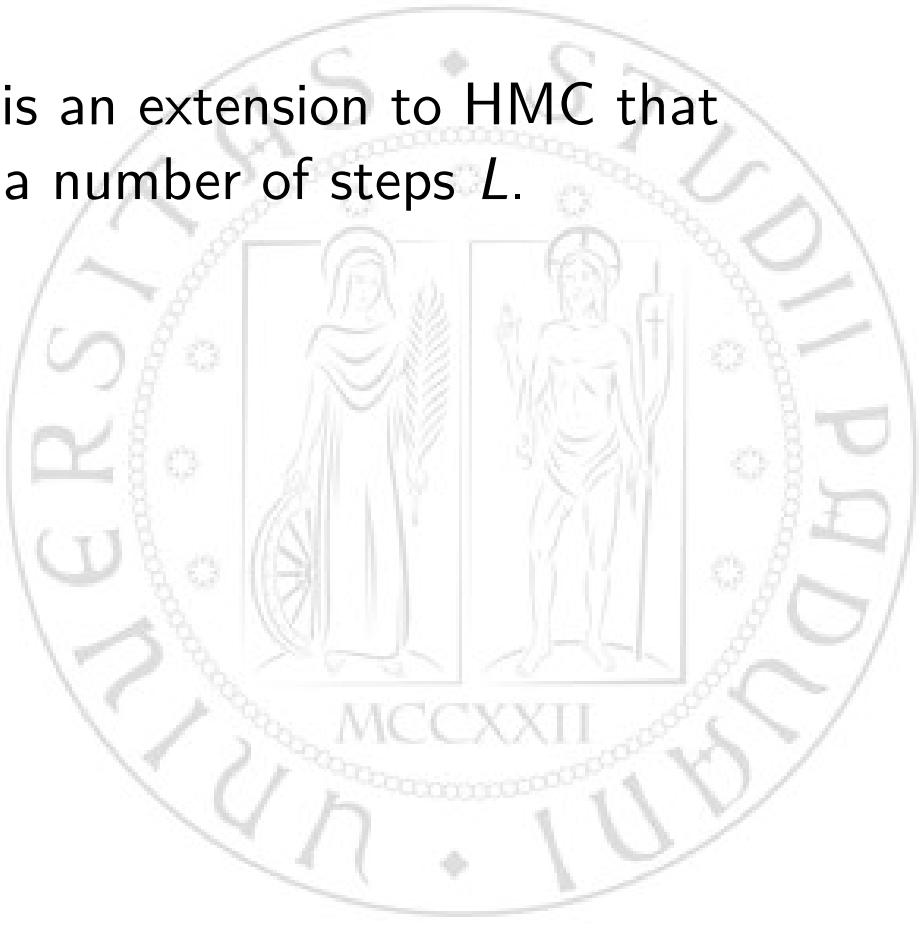
HMC is an *MCMC* algorithm that avoids random walk behaviour and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

No U-Turn Sampler

HMC is an *MCMC* algorithm that avoids random walk behaviour and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$$\Downarrow$$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.



No U-Turn Sampler

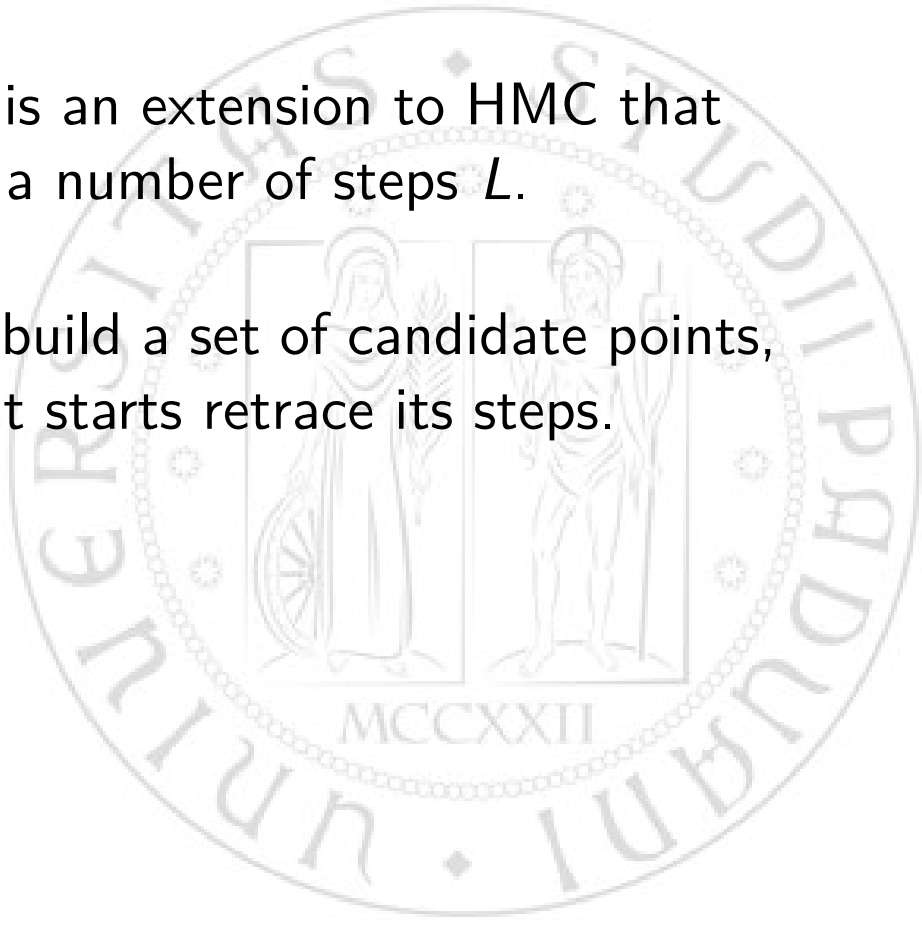
HMC is an *MCMC* algorithm that avoids random walk behaviour and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$\Downarrow$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.

$\Downarrow$

NUTS uses a recursive algorithm to build a set of candidate points, stopping automatically when it starts retrace its steps.



No U-Turn Sampler

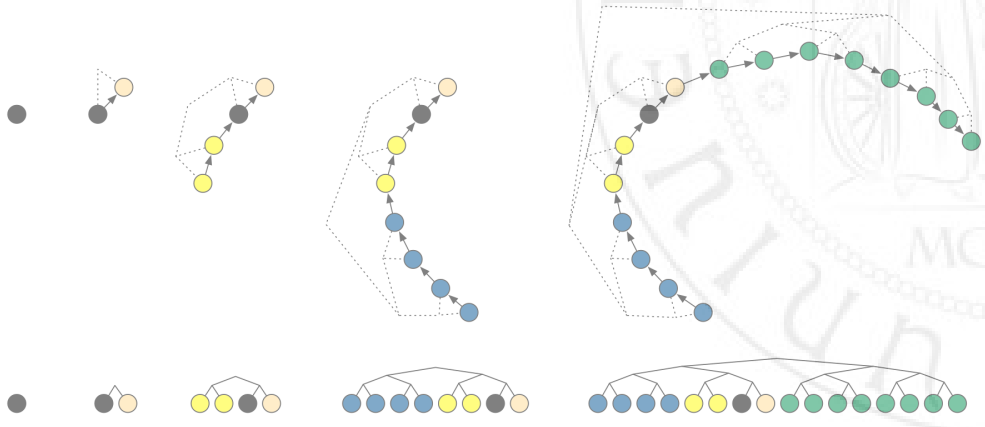
HMC is an *MCMC* algorithm that avoids random walk behaviour and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$\Downarrow$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.

$\Downarrow$

NUTS uses a recursive algorithm to build a set of candidate points, stopping automatically when it starts retrace its steps.



Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behaviour, but it comes with a price:

Hamiltonian Monte Carlo

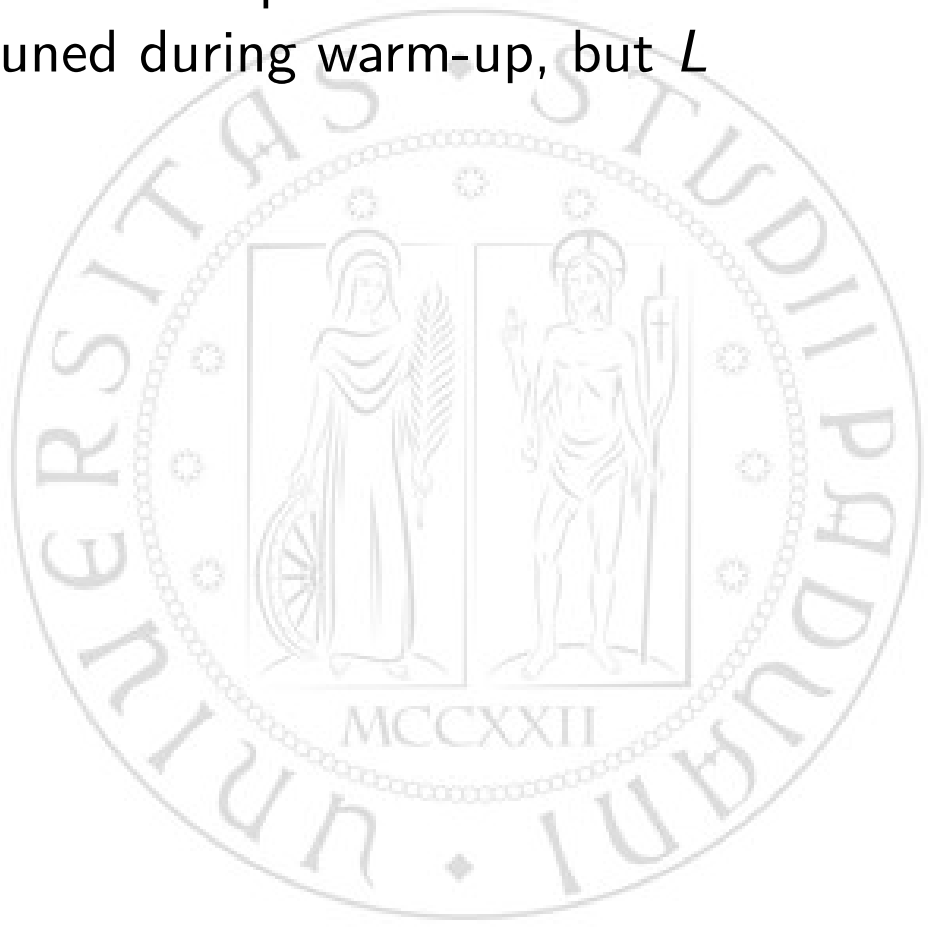
HMC generates proposals with high acceptance probability by suppressing the random walk behaviour, but it comes with a price:

- Computing the gradient of the log-posterior at each step.

Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behaviour, but it comes with a price:

- Computing the gradient of the log-posterior at each step.
- Tuning two user-specified parameters: step size ϵ and number of steps L. Tipically, ϵ can be tuned during warm-up, but L can require costly tuning runs.



Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behaviour, but it comes with a price:

- Computing the gradient of the log-posterior at each step.
- Tuning two user-specified parameters: step size ϵ and number of steps L. Tipically, ϵ can be tuned during warm-up, but L can require costly tuning runs.

Algorithm 1 Hamiltonian Monte Carlo

Given θ^0, ϵ, L, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Set $\theta^m \leftarrow \theta^{m-1}, \tilde{\theta} \leftarrow \theta^{m-1}, \tilde{r} \leftarrow r^0$.
  for $i = 1$ to L **do**
    Set $\tilde{\theta}, \tilde{r} \leftarrow \text{Leapfrog}(\tilde{\theta}, \tilde{r}, \epsilon)$.
  end for
  With probability $\alpha = \min\left\{1, \frac{\exp\{\mathcal{L}(\tilde{\theta}) - \frac{1}{2}\tilde{r}\cdot\tilde{r}\}}{\exp\{\mathcal{L}(\theta^{m-1}) - \frac{1}{2}r^0\cdot r^0\}}\right\}$, set $\theta^m \leftarrow \tilde{\theta}$, $r^m \leftarrow -\tilde{r}$.
end for

function Leapfrog(θ, r, ϵ)
Set $\tilde{r} \leftarrow r + (\epsilon/2)\nabla_\theta \mathcal{L}(\theta)$.
Set $\tilde{\theta} \leftarrow \theta + \epsilon\tilde{r}$.
Set $\tilde{r} \leftarrow \tilde{r} + (\epsilon/2)\nabla_\theta \mathcal{L}(\tilde{\theta})$.
return $\tilde{\theta}, \tilde{r}$.

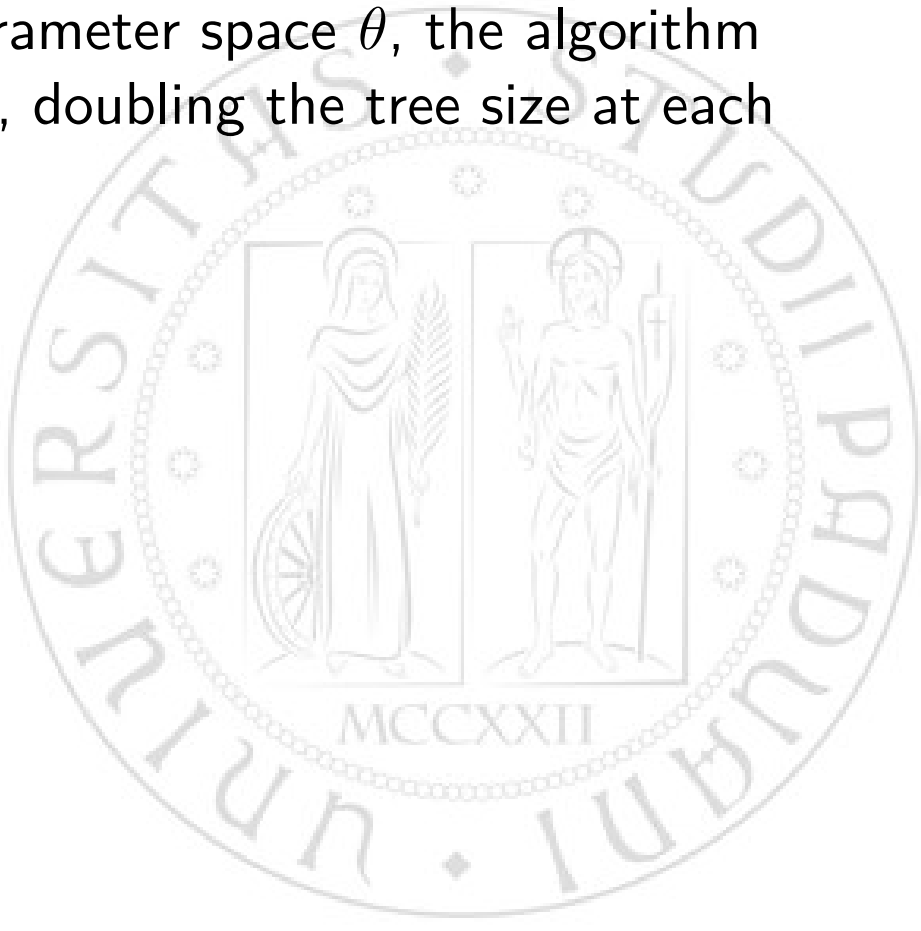
No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behaviour, but without the need to tune the number of Leap Frog steps L.

No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behaviour, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.



No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behaviour, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.
- To detect when to stop, the algorithm checks if the trajectory starts to turn back on itself: given a proposed point $\tilde{\theta}$, the algorithm checks if the dot product of the momentum vectors at θ and $\tilde{\theta}$ is negative.
  $(\theta - \tilde{\theta}) \cdot \frac{d}{dt}(\theta - \tilde{\theta}) = (\theta - \tilde{\theta}) \cdot \tilde{r} < 0$

No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behaviour, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.
- To detect when to stop, the algorithm checks if the trajectory starts to turn back on itself: given a proposed point $\tilde{\theta}$, the algorithm checks if the dot product of the momentum vectors at θ and $\tilde{\theta}$ is negative.
  $(\theta - \tilde{\theta}) \cdot \frac{d}{dt}(\theta - \tilde{\theta}) = (\theta - \tilde{\theta}) \cdot \tilde{r} < 0$
- To preserve time reversibility, NUTS runs the Hamiltonian dynamics in both forward and backward directions.

Probability slicing

To simplify the derivation and implementation of the algorithm, NUTS uses a slice variable u with conditional distribution

$$p(u|\theta, r) = \mathcal{U}(u; [0, exp(\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r)]).$$

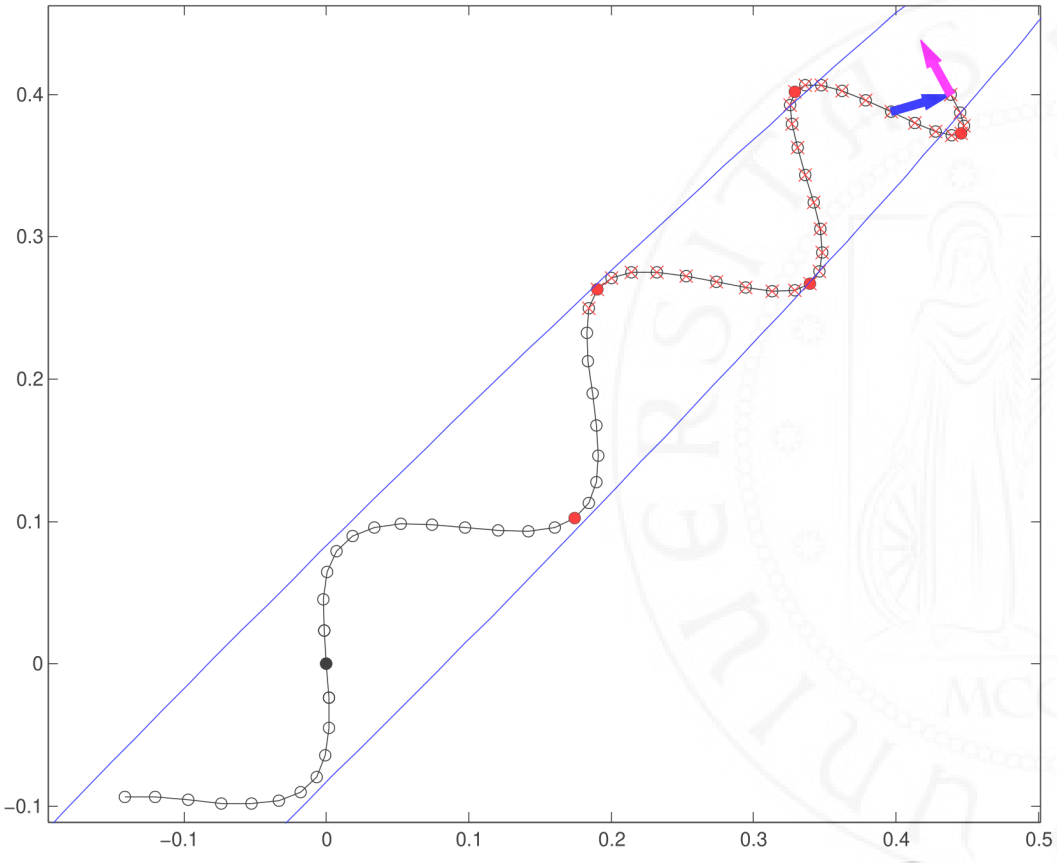
This way $p(\theta, r|u) = \mathcal{U}(\theta, r; \theta', r' | \exp(\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r) \geq u)$.

Probability slicing

To simplify the derivation and implementation of the algorithm, NUTS uses a slice variable u with conditional distribution

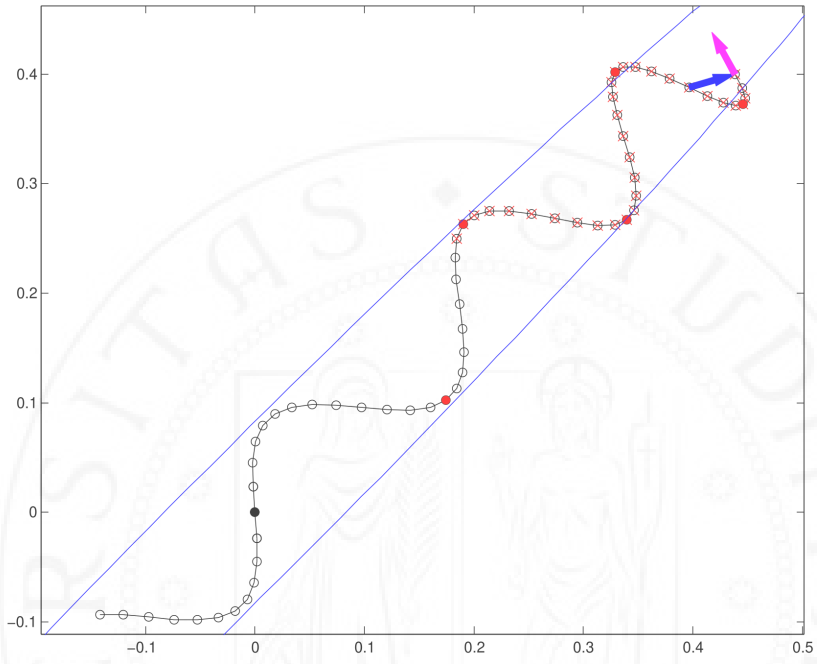
$$p(u|\theta, r) = \mathcal{U}(u; [0, exp(\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r)]).$$

This way $p(\theta, r|u) = \mathcal{U}(\theta, r; \theta', r' | \exp(\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r) \geq u)$.



High-level NUTS algorithm

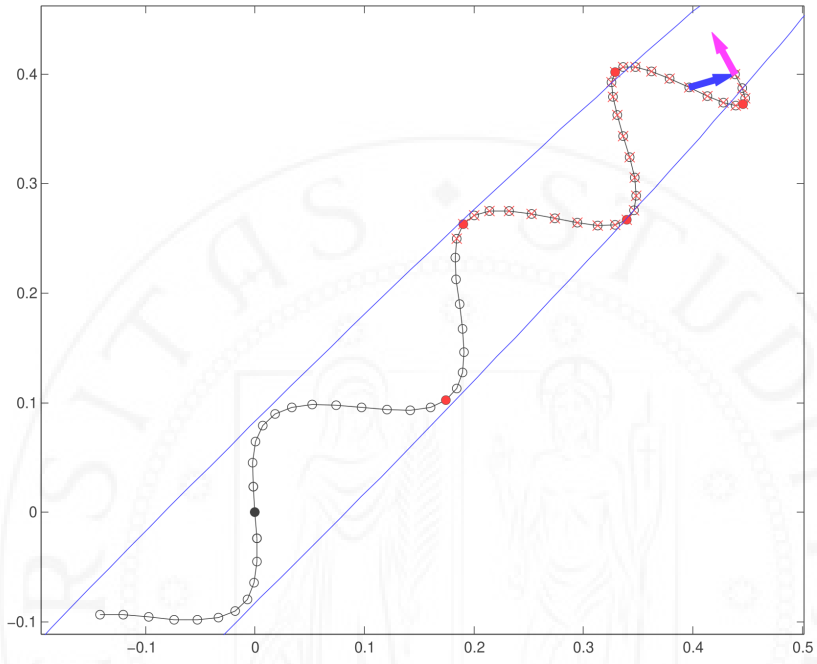
- The algorithm resamples $u|\theta, r$ at each iteration,



Example of a NUTS trajectory, with the slice variable u.

High-level NUTS algorithm

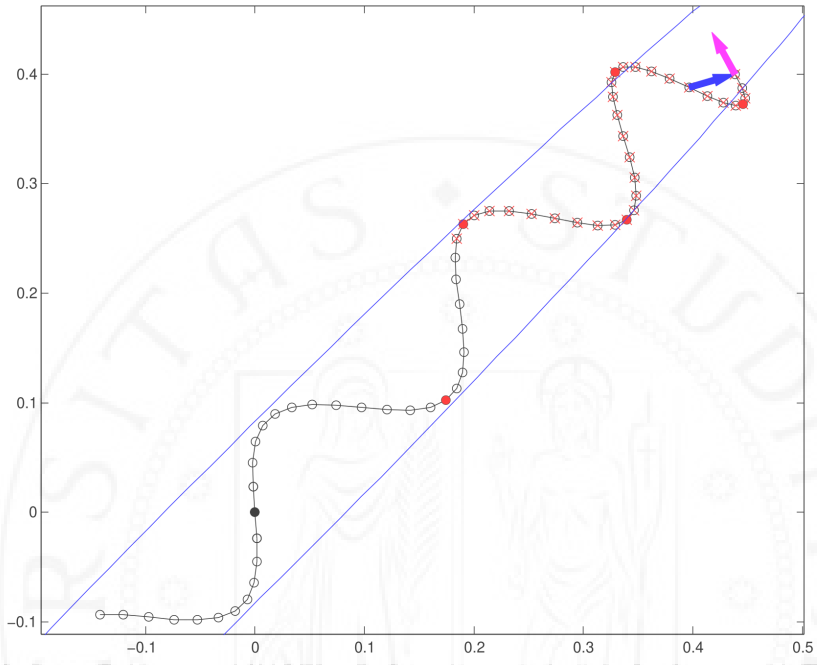
- The algorithm resamples $u|\theta, r$ at each iteration,
- uses Leapfrog integrator to trace a path forward and backwards first 1 step,



Example of a NUTS trajectory, with the slice variable u.

High-level NUTS algorithm

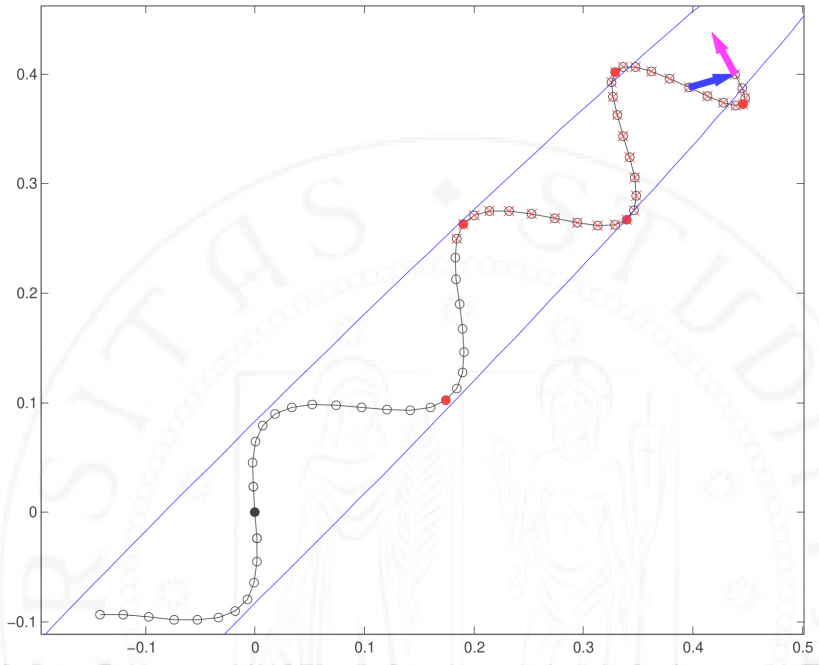
- The algorithm resamples $u|\theta, r$ at each iteration,
- uses Leapfrog integrator to trace a path forward and backwards first 1 step,
- then doubling for successive iterations.



Example of a NUTS trajectory, with the slice variable u.

High-level NUTS algorithm

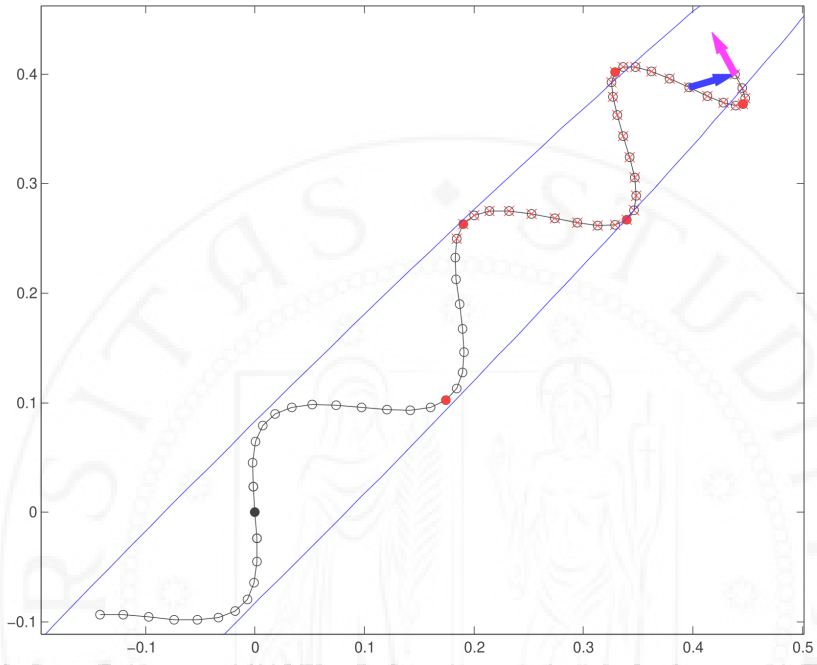
- The algorithm resamples $u|\theta, r$ at each iteration,
- uses Leapfrog integrator to trace a path forward and backwards first 1 step,
- then doubling for successive iterations.
- This doubling process implicitly builds a balanced binary tree whose leaf nodes correspond to position-momentum states.



Example of a NUTS trajectory, with the slice variable u.

High-level NUTS algorithm

- The algorithm resamples $u|\theta, r$ at each iteration,
- uses Leapfrog integrator to trace a path forward and backwards first 1 step,
- then doubling for successive iterations.
- This doubling process implicitly builds a balanced binary tree whose leaf nodes correspond to position-momentum states.
- The doubling is halted when the subtrajectory starts to double back on itself.



Example of a NUTS trajectory, with the slice variable u.

Detailed NUTS algorithm

Given $\mathcal{B}$, set of position-momentum states traced by the leapfrog integrator, we pick deterministically $\mathcal{C}$, the set of candidate points.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

Given $\mathcal{B}$, set of position-momentum states traced by the leapfrog integrator, we pick deterministically $\mathcal{C}$, the set of candidate points.

- Transitions between states in $\mathcal{C}$ must preserve proability volume and satisfy detailed balance.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

Given $\mathcal{B}$, set of position-momentum states traced by the leapfrog integrator, we pick deterministically $\mathcal{C}$, the set of candidate points.

- Transitions between states in $\mathcal{C}$ must preserve proability volume and satisfy detailed balance.
- The starting position-momentum state (θ, r) is added to $\mathcal{C}$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

Given $\mathcal{B}$, set of position-momentum states traced by the leapfrog integrator, we pick deterministically $\mathcal{C}$, the set of candidate points.

- Transitions between states in $\mathcal{C}$ must preserve proability volume and satisfy detailed balance.
- The starting position-momentum state (θ, r) is added to $\mathcal{C}$.
- Each point in $\mathcal{C}$ must satisfy the slice variable condition $exp\mathcal{L}(\theta) - \frac{1}{2} r \cdot r \geq u$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

Given $\mathcal{B}$, set of position-momentum states traced by the leapfrog integrator, we pick deterministically $\mathcal{C}$, the set of candidate points.

- Transitions between states in $\mathcal{C}$ must preserve proability volume and satisfy detailed balance.
- The starting position-momentum state (θ, r) is added to $\mathcal{C}$.
- Each point in $\mathcal{C}$ must satisfy the slice variable condition $exp\mathcal{L}(\theta) - \frac{1}{2} r \cdot r \geq u$.
- Each point in $\mathcal{C}$ must have the same probability of constructing the trajectory $\mathcal{B}$ and candidate set $\mathcal{C}$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

- Sample $r \sim \mathcal{N}(0, I)$,

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

- Sample $r \sim \mathcal{N}(0, I)$,
- $u \sim \mathcal{U}(0, exp\mathcal{L}(\theta^t) - \frac{1}{2} r \cdot r)$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

- Sample $r \sim \mathcal{N}(0, I)$,
- $u \sim \mathcal{U}(0, exp\mathcal{L}(\theta^t) - \frac{1}{2} r \cdot r)$.
- Sample $\mathcal{B}, \mathcal{C} \sim p(\mathcal{B}, \mathcal{C} | \theta^t, r, u, \epsilon)$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

- Sample $r \sim \mathcal{N}(0, I)$,
- $u \sim \mathcal{U}(0, exp\mathcal{L}(\theta^t) - \frac{1}{2} r \cdot r)$.
- Sample $\mathcal{B}, \mathcal{C} \sim p(\mathcal{B}, \mathcal{C} | \theta^t, r, u, \epsilon)$.
- Sample $\theta^{t+1}, r \sim T(\theta^t, r, \mathcal{C})$, transition kernel.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

We can ensure that with the following procedure at iteration t:

- Sample $r \sim \mathcal{N}(0, I)$,
- $u \sim \mathcal{U}(0, exp\mathcal{L}(\theta^t) - \frac{1}{2} r \cdot r)$.
- Sample $\mathcal{B}, \mathcal{C} \sim p(\mathcal{B}, \mathcal{C} | \theta^t, r, u, \epsilon)$.
- Sample $\theta^{t+1}, r \sim T(\theta^t, r, \mathcal{C})$, transition kernel.

Where the transition kernel $T(\theta, r, \mathcal{C})$ has left the distribution over $\mathcal{C}$ invariant, valid because ϵ is uniform on the elements of $\mathcal{C}$ and $p(\theta, r | u, \mathcal{B}, \mathcal{C}, \epsilon) \propto \mathbb{I}[(\theta, r) \in \mathcal{C}]$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

- First a single node for the starting state.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

- First a single node for the starting state.
- Each iteration j preoceeds with in a direction $v_j \sim \mathcal{U}(-1, 1)$ taking 2^j Leapfrog steps.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

- First a single node for the starting state.
- Each iteration j preoceeds with in a direction $v_j \sim \mathcal{U}(-1, 1)$ taking 2^j Leapfrog steps.
- The iteration stops when the trajectory starts to double back on itself

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

- First a single node for the starting state.
- Each iteration j preoceeds with in a direction $v_j \sim \mathcal{U}(-1, 1)$ taking 2^j Leapfrog steps.
- The iteration stops when the trajectory starts to double back on itself
- or when the error in the simulation becomes too large, $\mathcal{L}(\theta) - \frac{1}{2} r \cdot r - \log u < -\Delta_{max}$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

The trajectory $\mathcal{B}$ is built with a doubling binary tree structure:

- First a single node for the starting state.
- Each iteration j preoceeds with in a direction $v_j \sim \mathcal{U}(-1, 1)$ taking 2^j Leapfrog steps.
- The iteration stops when the trajectory starts to double back on itself
- or when the error in the simulation becomes too large, $\mathcal{L}(\theta) - \frac{1}{2} r \cdot r - \log u < -\Delta_{max}$.

The doubling process defines the distribution $p(\mathcal{B}|\theta^t, r, u, \epsilon)$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

- NUTS algorithm repeatedly calls the BuildTree function, returning left and right endpoints of the trajectory, and all visited states $\mathcal{C}'$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

- NUTS algorithm repeatedly calls the BuildTree function, returning left and right endpoints of the trajectory, and all visited states $\mathcal{C}'$.
- If one of the two stepping conditions is met, the algorithm terminates and the new candidates $\mathcal{C}'$ are rejected.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

- NUTS algorithm repeatedly calls the BuildTree function, returning left and right endpoints of the trajectory, and all visited states $\mathcal{C}'$.
- If one of the two stepping conditions is met, the algorithm terminates and the new candidates $\mathcal{C}'$ are rejected.
- Otherwise, the new candidates $\mathcal{C}'$ are added to the set of candidate points $\mathcal{C}$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

- NUTS algorithm repeatedly calls the BuildTree function, returning left and right endpoints of the trajectory, and all visited states $\mathcal{C}'$.
- If one of the two stepping conditions is met, the algorithm terminates and the new candidates $\mathcal{C}'$ are rejected.
- Otherwise, the new candidates $\mathcal{C}'$ are added to the set of candidate points $\mathcal{C}$.
- If a global U turn condition is met, the doubling process is terminated.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Detailed NUTS algorithm

- NUTS algorithm repeatedly calls the BuildTree function, returning left and right endpoints of the trajectory, and all visited states $\mathcal{C}'$.
- If one of the two stepping conditions is met, the algorithm terminates and the new candidates $\mathcal{C}'$ are rejected.
- Otherwise, the new candidates $\mathcal{C}'$ are added to the set of candidate points $\mathcal{C}$.
- If a global U turn condition is met, the doubling process is terminated.
- The new position-momentum state (θ^{t+1}, r) is sampled from the set of candidate points $\mathcal{C}$.

Algorithm 2 Naive No-U-Turn Sampler

Given θ^0, ϵ, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Resample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0$, $r^+ = r^0$, $j = 0$, $\mathcal{C} = \{(\theta^{m-1}, r^0)\}$, $s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon)$.
    else
      $-, -, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon)$.
    end if
    if $s' = 1$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$.
    end if
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  Sample θ^m, r uniformly at random from $\mathcal{C}$.
end for

function BuildTree$(\theta, r, u, v, j, \epsilon)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $\mathcal{C}' \leftarrow \begin{cases} \{(\theta', r')\} & \text{if } u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\} \\ \emptyset & \text{else} \end{cases}$
  $s' \leftarrow \mathbb{I}[\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' > \log u - \Delta_{\max}]$.
  return $\theta', r', \theta', r', \mathcal{C}', s'$.
else
  Recursion—build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s' \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon)$.
  if $v = -1$ **then**
    $\theta^-, r^-, -, -, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon)$.
  else
    $-, -, \theta^+, r^+, \mathcal{C}'', s'' \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon)$.
  end if
  $s' \leftarrow s' s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \mathcal{C}''$.
  return $\theta^-, r^-, \theta^+, r^+, \mathcal{C}', s'$.
end if

Optimized NUTS algorithm

Some optimizations can be made to the NUTS algorithm:

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0$, $j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa} \log \epsilon_m + (1 - m^{-\kappa}) \log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

function $\text{BuildTree}(\theta, r, u, v, j, \epsilon, \theta^0, r^0)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $n' \leftarrow \mathbb{I}[u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  $s' \leftarrow \mathbb{I}[u < \exp\{\Delta_{\max} + \mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  return $\theta', r', \theta', r', \theta', n', s', \min\{1, \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' - \mathcal{L}(\theta^0) + \frac{1}{2} r^0 \cdot r^0\}\}, 1$.
else
  Recursion—implicitly build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon, \theta^0, r^0)$.
  if $s' = 1$ **then**
    if $v = -1$ **then**
      $\theta^-, r^-, -, -, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    end if
    With probability $\frac{n''}{n' + n''}$, set $\theta' \leftarrow \theta''$.
    Set $\alpha' \leftarrow \alpha' + \alpha''$, $n'_\alpha \leftarrow n'_\alpha + n''_\alpha$.
    $s' \leftarrow s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$
    $n' \leftarrow n' + n''$
  end if
  return $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha$.
end if

Optimized NUTS algorithm

Some optimizations can be made to the NUTS algorithm:

- Early stopping when a stopping criterion is met mid-process.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0$, $j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa} \log \epsilon_m + (1 - m^{-\kappa}) \log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

function $\text{BuildTree}(\theta, r, u, v, j, \epsilon, \theta^0, r^0)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $n' \leftarrow \mathbb{I}[u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  $s' \leftarrow \mathbb{I}[u < \exp\{\Delta_{\max} + \mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  return $\theta', r', \theta', r', \theta', n', s', \min\{1, \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' - \mathcal{L}(\theta^0) + \frac{1}{2} r^0 \cdot r^0\}\}, 1$.
else
  Recursion—implicitly build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon, \theta^0, r^0)$.
  if $s' = 1$ **then**
    if $v = -1$ **then**
      $\theta^-, r^-, -, -, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    end if
    With probability $\frac{n''}{n' + n''}$, set $\theta' \leftarrow \theta''$.
    Set $\alpha' \leftarrow \alpha' + \alpha''$, $n'_\alpha \leftarrow n'_\alpha + n''_\alpha$.
    $s' \leftarrow s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$
    $n' \leftarrow n' + n''$
  end if
  return $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha$.
end if

Optimized NUTS algorithm

Some optimizations can be made to the NUTS algorithm:

- Early stopping when a stopping criterion is met mid-process.
- Single Proposed State kept in memory, by sampling a candidate for each subtree and drawing from them using their relative weights (i.e. the size of the subtree).

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0$, $j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1} \theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa} \log \epsilon_m + (1 - m^{-\kappa}) \log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

function $\text{BuildTree}(\theta, r, u, v, j, \epsilon, \theta^0, r^0)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $n' \leftarrow \mathbb{I}[u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  $s' \leftarrow \mathbb{I}[u < \exp\{\Delta_{\max} + \mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  return $\theta', r', \theta', r', \theta', n', s', \min\{1, \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' - \mathcal{L}(\theta^0) + \frac{1}{2} r^0 \cdot r^0\}\}, 1$.
else
  Recursion—implicitly build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon, \theta^0, r^0)$.
  if $s' = 1$ **then**
    if $v = -1$ **then**
      $\theta^-, r^-, -, -, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    end if
    With probability $\frac{n''}{n' + n''}$, set $\theta' \leftarrow \theta''$.
    Set $\alpha' \leftarrow \alpha' + \alpha''$, $n'_\alpha \leftarrow n'_\alpha + n''_\alpha$.
    $s' \leftarrow s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$
    $n' \leftarrow n' + n''$
  end if
  return $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha$.
end if

Optimized NUTS algorithm

Some optimizations can be made to the NUTS algorithm:

- Early stopping when a stopping criterion is met mid-process.
- Single Proposed State kept in memory, by sampling a candidate for each subtree and drawing from them using their relative weights (i.e. the size of the subtree).
- Adaptive tuning of the step size parameter ϵ, using dual averaging.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2} r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0$, $j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1} \theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa} \log \epsilon_m + (1 - m^{-\kappa}) \log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

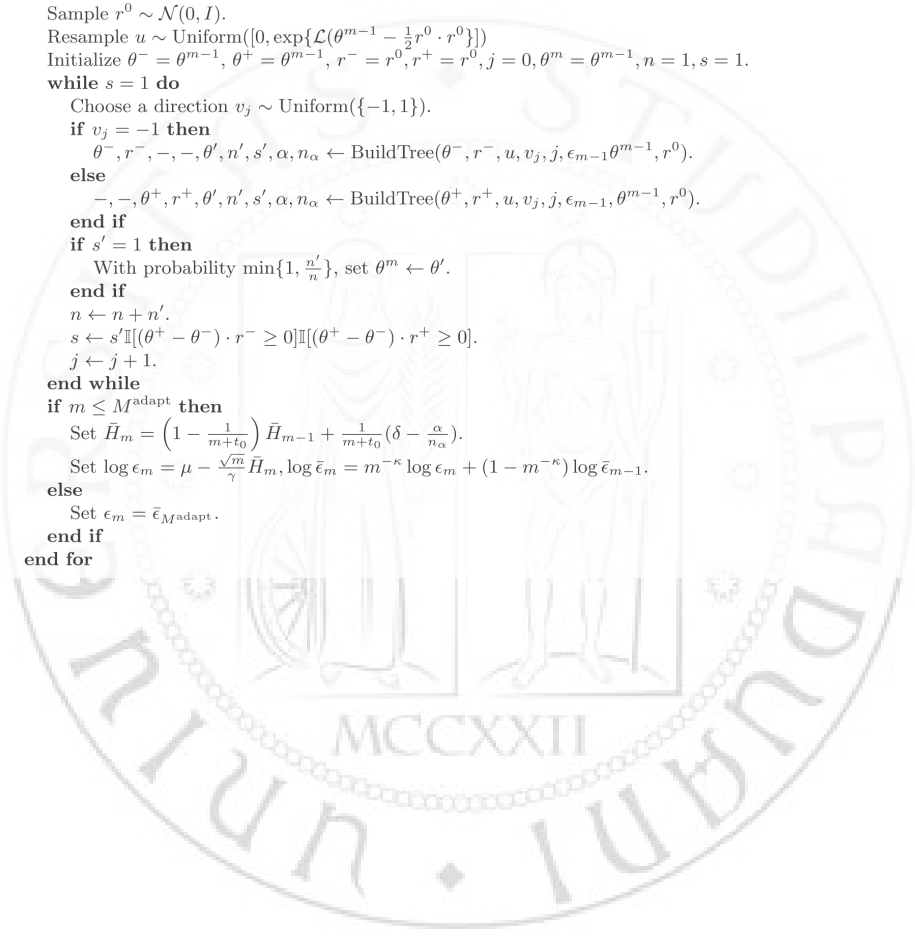
function $\text{BuildTree}(\theta, r, u, v, j, \epsilon, \theta^0, r^0)$
if $j = 0$ **then**
  Base case—take one leapfrog step in the direction v.
  $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, v\epsilon)$.
  $n' \leftarrow \mathbb{I}[u \leq \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  $s' \leftarrow \mathbb{I}[u < \exp\{\Delta_{\max} + \mathcal{L}(\theta') - \frac{1}{2} r' \cdot r'\}]$.
  return $\theta', r', \theta', r', \theta', n', s', \min\{1, \exp\{\mathcal{L}(\theta') - \frac{1}{2} r' \cdot r' - \mathcal{L}(\theta^0) + \frac{1}{2} r^0 \cdot r^0\}\}, 1$.
else
  Recursion—implicitly build the left and right subtrees.
  $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon, \theta^0, r^0)$.
  if $s' = 1$ **then**
    if $v = -1$ **then**
      $\theta^-, r^-, -, -, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon, \theta^0, r^0)$.
    end if
    With probability $\frac{n''}{n' + n''}$, set $\theta' \leftarrow \theta''$.
    Set $\alpha' \leftarrow \alpha' + \alpha''$, $n'_\alpha \leftarrow n'_\alpha + n''_\alpha$.
    $s' \leftarrow s'' \mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0] \mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$
    $n' \leftarrow n' + n''$
  end if
  return $\theta^-, r^-, \theta^+, r^+, \theta', n', s', \alpha', n'_\alpha$.
end if

Dual Averaging for ϵ

ϵ is set using stochastic optimization with vanishing adaptation.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta), \mu = \log(10\epsilon_0), \bar{\epsilon}_0 = 1, \bar{H}_0 = 0, \gamma = 0.05, t_0 = 10, \kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2}r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0, j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s'\mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0]\mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m+t_0}\right)\bar{H}_{m-1} + \frac{1}{m+t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\bar{H}_m, \log \bar{\epsilon}_m = m^{-\kappa} \log \epsilon_m + (1 - m^{-\kappa}) \log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for



Dual Averaging for ϵ

ϵ is set using stochastic optimization with vanishing adaptation.

- Defining an expectation $h(\epsilon) = \mathbb{E}_t[H_t|x] = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_t[H_t|x]$.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta), \mu = \log(10\epsilon_0), \bar{\epsilon}_0 = 1, \bar{H}_0 = 0, \gamma = 0.05, t_0 = 10, \kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2}r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0, j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s'\mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0]\mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m+t_0}\right)\bar{H}_{m-1} + \frac{1}{m+t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\bar{H}_m, \log \bar{\epsilon}_m = m^{-\kappa}\log \epsilon_m + (1 - m^{-\kappa})\log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

Dual Averaging for ϵ

ϵ is set using stochastic optimization with vanishing adaptation.

- Defining an expectation $h(\epsilon) = \mathbb{E}_t[H_t|x] = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_t[H_t|x]$.
- Using the desired acceptance probability δ to define the error $H_t = \delta - \alpha_t$, where α_t is the acceptance probability at iteration t.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta), \mu = \log(10\epsilon_0), \bar{\epsilon}_0 = 1, \bar{H}_0 = 0, \gamma = 0.05, t_0 = 10, \kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2}r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0, j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s'\mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0]\mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m+t_0}\right)\bar{H}_{m-1} + \frac{1}{m+t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\bar{H}_m, \log \bar{\epsilon}_m = m^{-\kappa}\log\epsilon_m + (1 - m^{-\kappa})\log\bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

Dual Averaging for ϵ

ϵ is set using stochastic optimization with vanishing adaptation.

- Defining an expectation $h(\epsilon) = \mathbb{E}_t[H_t|x] = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_t[H_t|x]$.
- Using the desired acceptance probability δ to define the error $H_t = \delta - \alpha_t$, where α_t is the acceptance probability at iteration t.
- Updating the parameter with the statistics H_t: $x_{t+1} \leftarrow \mu - \frac{\sqrt{t}}{\gamma}\frac{1}{t+t_0}\sum_{i=1}^{t} H_i$, $\bar{x}_{t+1} \leftarrow \bar{x}_t + \eta_t x_{t+1} + (1-\eta_t)\bar{x}_t$ with $\eta_t = t^{-k}$.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2}r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0, j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s'\mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0]\mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m+t_0}\right)\bar{H}_{m-1} + \frac{1}{m+t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa}\log \epsilon_m + (1 - m^{-\kappa})\log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

Dual Averaging for ϵ

ϵ is set using stochastic optimization with vanishing adaptation.

- Defining an expectation $h(\epsilon) = \mathbb{E}_t[H_t|x] = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_t[H_t|x]$.
- Using the desired acceptance probability δ to define the error $H_t = \delta - \alpha_t$, where α_t is the acceptance probability at iteration t.
- Updating the parameter with the statistics H_t: $x_{t+1} \leftarrow \mu - \frac{\sqrt{t}}{\gamma}\frac{1}{t+t_0}\sum_{i=1}^{t} H_i$, $\bar{x}_{t+1} \leftarrow \bar{x}_t + \eta_t x_{t+1} + (1-\eta_t)\bar{x}_t$ with $\eta_t = t^{-k}$.

Algorithm 6 No-U-Turn Sampler with Dual Averaging

Given θ^0, δ, $\mathcal{L}$, M, M^{adapt}:
Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$, $\mu = \log(10\epsilon_0)$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$.
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Resample $u \sim \text{Uniform}([0, \exp\{\mathcal{L}(\theta^{m-1} - \frac{1}{2}r^0 \cdot r^0\}])$
  Initialize $\theta^- = \theta^{m-1}$, $\theta^+ = \theta^{m-1}$, $r^- = r^0, r^+ = r^0, j = 0, \theta^m = \theta^{m-1}, n = 1, s = 1$.
  while $s = 1$ **do**
    Choose a direction $v_j \sim \text{Uniform}(\{-1, 1\})$.
    if $v_j = -1$ **then**
      $\theta^-, r^-, -, -, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v_j, j, \epsilon_{m-1}\theta^{m-1}, r^0)$.
    else
      $-, -, \theta^+, r^+, \theta', n', s', \alpha, n_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v_j, j, \epsilon_{m-1}, \theta^{m-1}, r^0)$.
    end if
    if $s' = 1$ **then**
      With probability $\min\{1, \frac{n'}{n}\}$, set $\theta^m \leftarrow \theta'$.
    end if
    $n \leftarrow n + n'$.
    $s \leftarrow s'\mathbb{I}[(\theta^+ - \theta^-) \cdot r^- \geq 0]\mathbb{I}[(\theta^+ - \theta^-) \cdot r^+ \geq 0]$.
    $j \leftarrow j + 1$.
  end while
  if $m \leq M^{\text{adapt}}$ **then**
    Set $\bar{H}_m = \left(1 - \frac{1}{m+t_0}\right)\bar{H}_{m-1} + \frac{1}{m+t_0}(\delta - \frac{\alpha}{n_\alpha})$.
    Set $\log \epsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\bar{H}_m$, $\log \bar{\epsilon}_m = m^{-\kappa}\log \epsilon_m + (1 - m^{-\kappa})\log \bar{\epsilon}_{m-1}$.
  else
    Set $\epsilon_m = \bar{\epsilon}_{M^{\text{adapt}}}$.
  end if
end for

$\mu = \log 10\epsilon_1$ is a freely chosen point, $\gamma = 0.05$ is a free parameter for the speed of convergence, and $t_0 = 10$ is a free parameter that stabilizes the initial iterations and $k = 0.75 \in (0.5, 1]$.

Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1$.
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.

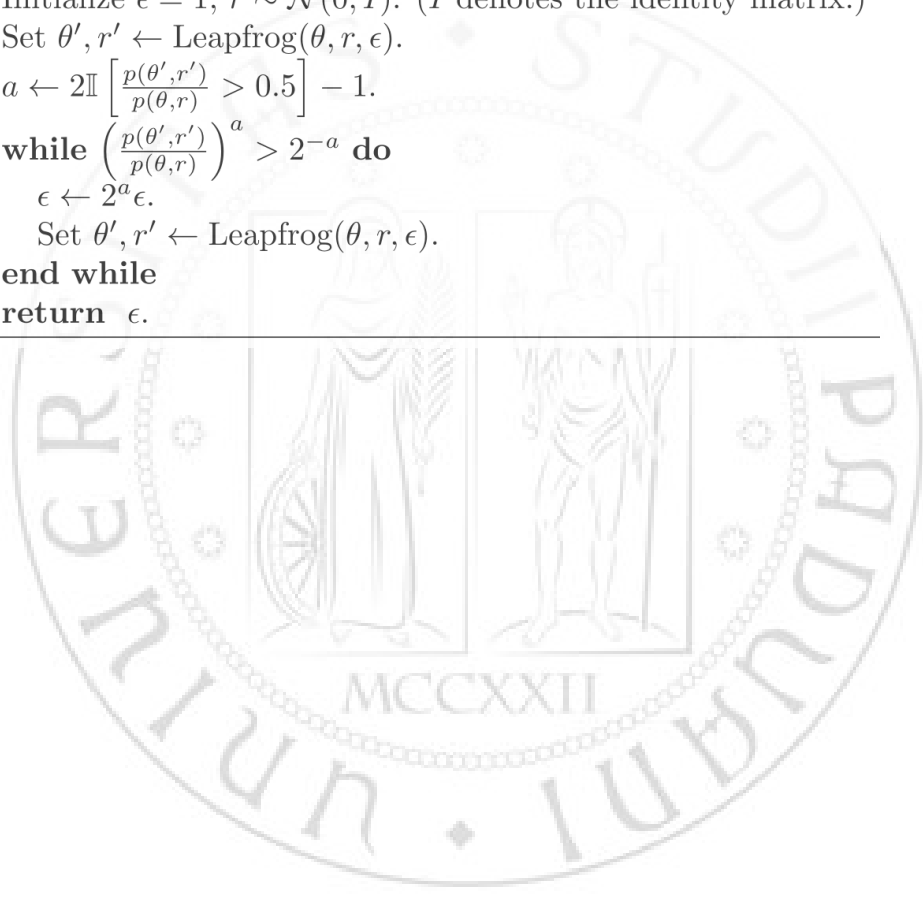
Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

- This heuristic doubles or halves the value of ϵ_1 until the acceptance probability of the Langevin (Leapfrog) proposal crosses 0.5.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1.$
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.



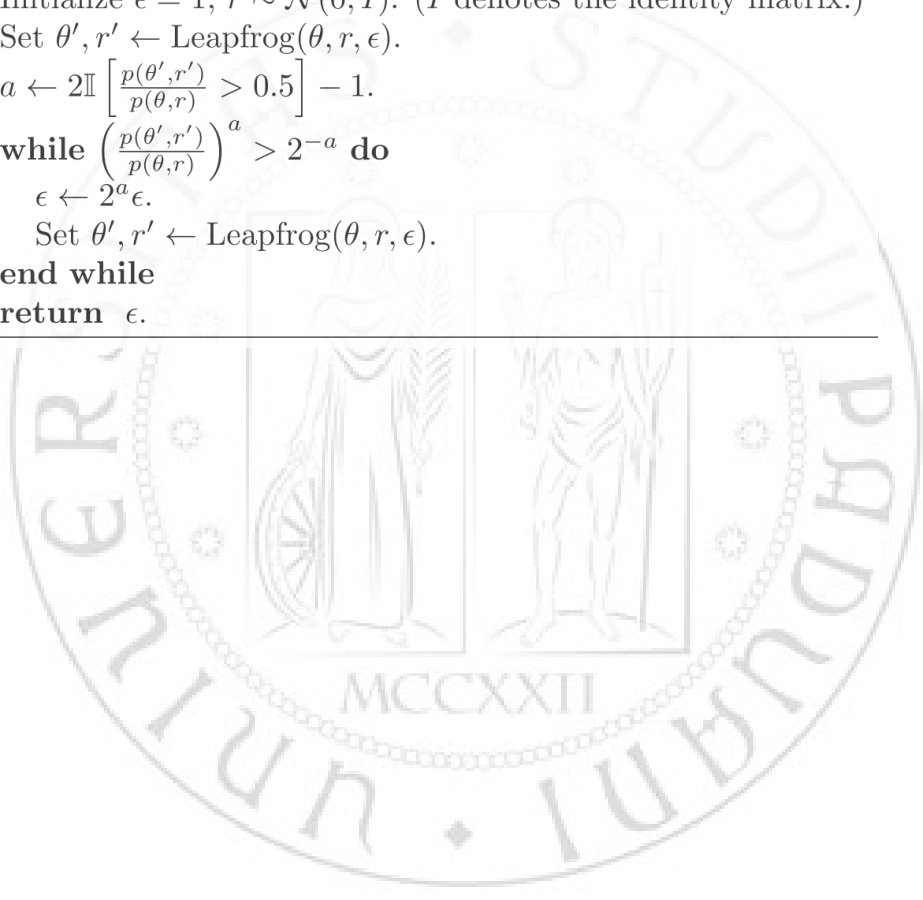
Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

- This heuristic doubles or halves the value of ϵ_1 until the acceptance probability of the Langevin (Leapfrog) proposal crosses 0.5.
- NUTS does not have a single acceptance probability, so we use an average HMC acceptance probability over the final doubling iteration states.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1.$
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.



Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

- This heuristic doubles or halves the value of ϵ_1 until the acceptance probability of the Langevin (Leapfrog) proposal crosses 0.5.
- NUTS does not have a single acceptance probability, so we use an average HMC acceptance probability over the final doubling iteration states.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1.$
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.

$$H^{NUTS} = \frac{1}{|\mathcal{B}_t^f|} \sum_{\theta, r \in \mathcal{B}_t^f} \min 1, \frac{\pi(\theta, r)}{\pi(\theta_{t-1}, r_t^0)}.$$

Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

- This heuristic doubles or halves the value of ϵ_1 until the acceptance probability of the Langevin (Leapfrog) proposal crosses 0.5.
- NUTS does not have a single acceptance probability, so we use an average HMC acceptance probability over the final doubling iteration states.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1.$
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.

$$H^{NUTS} = \frac{1}{|\mathcal{B}_t^f|} \sum_{\theta, r \in \mathcal{B}_t^f} \min 1, \frac{\pi(\theta, r)}{\pi(\theta_{t-1}, r_t^0)}.$$

We can use Dual Averaging with $H_t = \delta - H^{NUTS}$ and $x_t = \log \epsilon_t$ to coerce the average of H^{NUTS} to δ.

Heuristic function for ϵ

Convergence of Dual Averaging for ϵ is faster starting from a reasonable setting of parameters.

- This heuristic doubles or halves the value of ϵ_1 until the acceptance probability of the Langevin (Leapfrog) proposal crosses 0.5.
- NUTS does not have a single acceptance probability, so we use an average HMC acceptance probability over the final doubling iteration states.

Algorithm 4 Heuristic for choosing an initial value of ϵ

function FindReasonableEpsilon(θ)
Initialize $\epsilon = 1$, $r \sim \mathcal{N}(0, I)$. (I denotes the identity matrix.)
Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
$a \leftarrow 2\mathbb{I}\left[\frac{p(\theta', r')}{p(\theta, r)} > 0.5\right] - 1.$
while $\left(\frac{p(\theta', r')}{p(\theta, r)}\right)^a > 2^{-a}$ **do**
  $\epsilon \leftarrow 2^a \epsilon$.
  Set $\theta', r' \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$.
end while
return ϵ.

$$H^{NUTS} = \frac{1}{|\mathcal{B}_t^f|} \sum_{\theta, r \in \mathcal{B}_t^f} \min 1, \frac{\pi(\theta, r)}{\pi(\theta_{t-1}, r_t^0)}.$$

We can use Dual Averaging with $H_t = \delta - H^{NUTS}$ and $x_t = \log \epsilon_t$ to coerce the average of H^{NUTS} to δ.

Now the algorithm only requires an acceptance probability δ as input, and a number of iterations M^{adapt} for the adaptation phase.

NUTS implementation

In order to implement the algorithm we need some tools:

- Automatic differentiation
- Vectorial computation for high dimension spaces
- Random number generator
- Just in time compilation (Not needed but useful...)

NUTS implementation

In order to implement the algorithm we need some tools:

- Automatic differentiation
- Vectorial computation for high dimension spaces
- Random number generator
- Just in time compilation (Not needed but useful...)

Python vs JAX

Python vs JAX

Python (Recursive)

- Natural use of recursion.
- Relies on the call stack.
- Allows side effects.
- Mutable values.
- Dynamic control flow.
- Execution not compilable.

Python vs JAX

Python (Recursive)

- Natural use of recursion.
- Relies on the call stack.
- Allows side effects.
- Mutable values.
- Dynamic control flow.
- Execution not compilable.

JAX (Iterative & Functional)

- Recursion not supported by XLA.
- Functional loops (e.g., `lax.fori_loop`).
- Pure functions: no side effects.
- Immutable values.
- Static control flow for compilation.
- Works with `jit`, `vmap`, `grad`.

Recursion Vs Iteration

Algorithm 1: recursion

Input: n

Output: $n!$

if $n \leq 1$ **then**

  return *1*

else

  return $n \cdot$ *factorial*$(n-1)$

Algorithm 2: iteration

Input: n

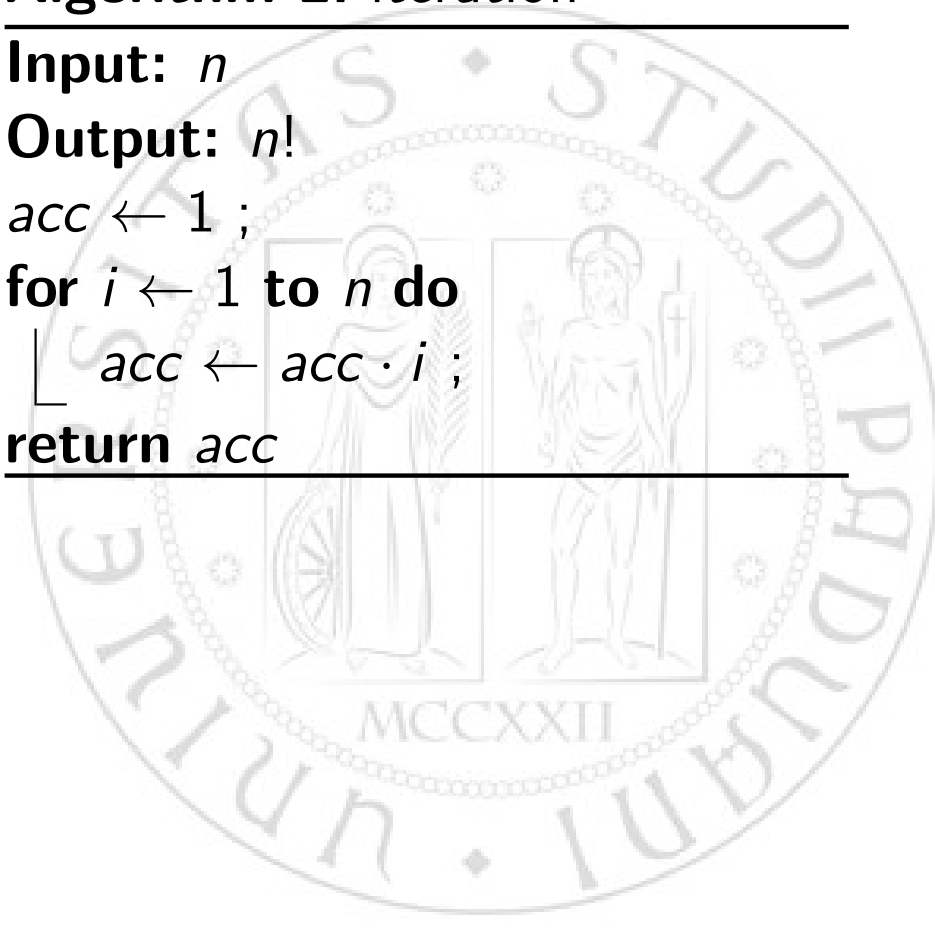
Output: $n!$

$acc \leftarrow 1$;

for $i \leftarrow 1$ **to** n **do**

  $acc \leftarrow acc \cdot i$;

return *acc*



Code implementation

```python
# Python factorial
def factorial_recursive(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)
```

Code implementation

```python
# Python factorial
def factorial_recursive(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)
```

```python
# JAX factorial
import jax
import jax.numpy as jnp
from jax import lax

def factorial_jax(n: int):
    def body(i, acc):
        return acc * (i + 1)
    return lax.fori_loop(0, n, body, 1)
```

NUTS Architecture Overview — Part I

1. Base Components

`log_p`: Joint log-density $L(\theta) - \frac{1}{2} r^T r$.
`stop_criterion`: Detects U-turns to stop tree expansion.
`Leapfrog`: Hamiltonian integrator (reversible, volume-preserving).

2. Tree Construction

`build_tree_base`: Depth-0 subtree via one leapfrog step.
`BuildTree`: Recursive tree expansion using `lax.fori_loop`; aggregates valid states, acceptance stats, and U-turn checks.

3. NUTS Kernel

`NUTS_one_step`: Full NUTS transition (Algorithm 3).
Samples momentum, performs slice sampling, builds the tree, selects candidate state, computes acceptance rate.

NUTS Architecture Overview — Part II

4. Step Size Initialization & Adaptation

`FindReasonableEpsilon`: Heuristic search for initial step size.
`dual_avg_init`: Initializes dual-averaging parameters.
`dual_avg_update`: Adapts step size toward target acceptance.

5. Warmup Phase

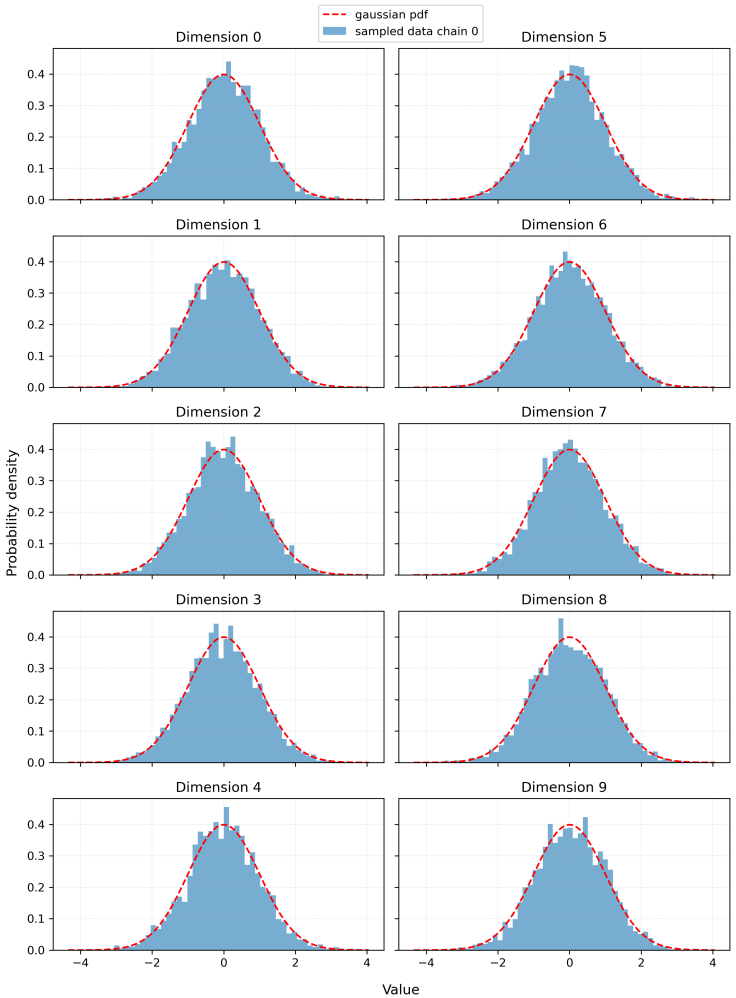
`NUTS_warmup`: Runs warmup (Algorithm 6), updating dual averaging and returning tuned step size ϵ_{final}.
`WarmupConfig`: Stores warmup settings.

6. Full Sampler

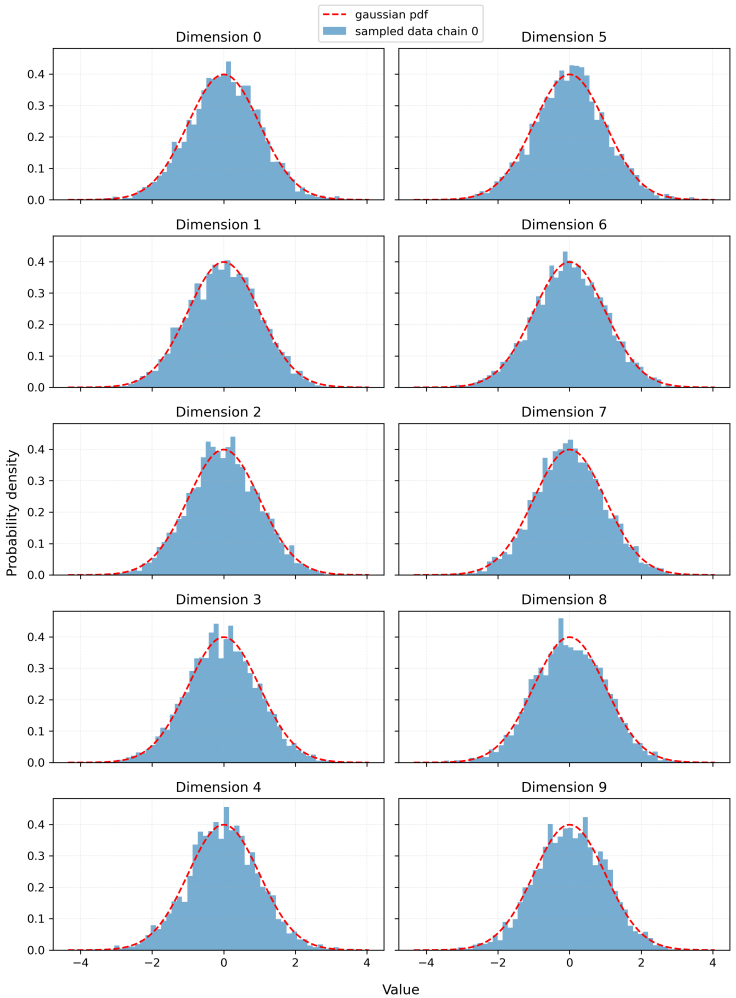
`nuts_sampler`: NUTS without warmup.
`NUTS_sampler_wu`: Full sampler with warmup + sampling phase.

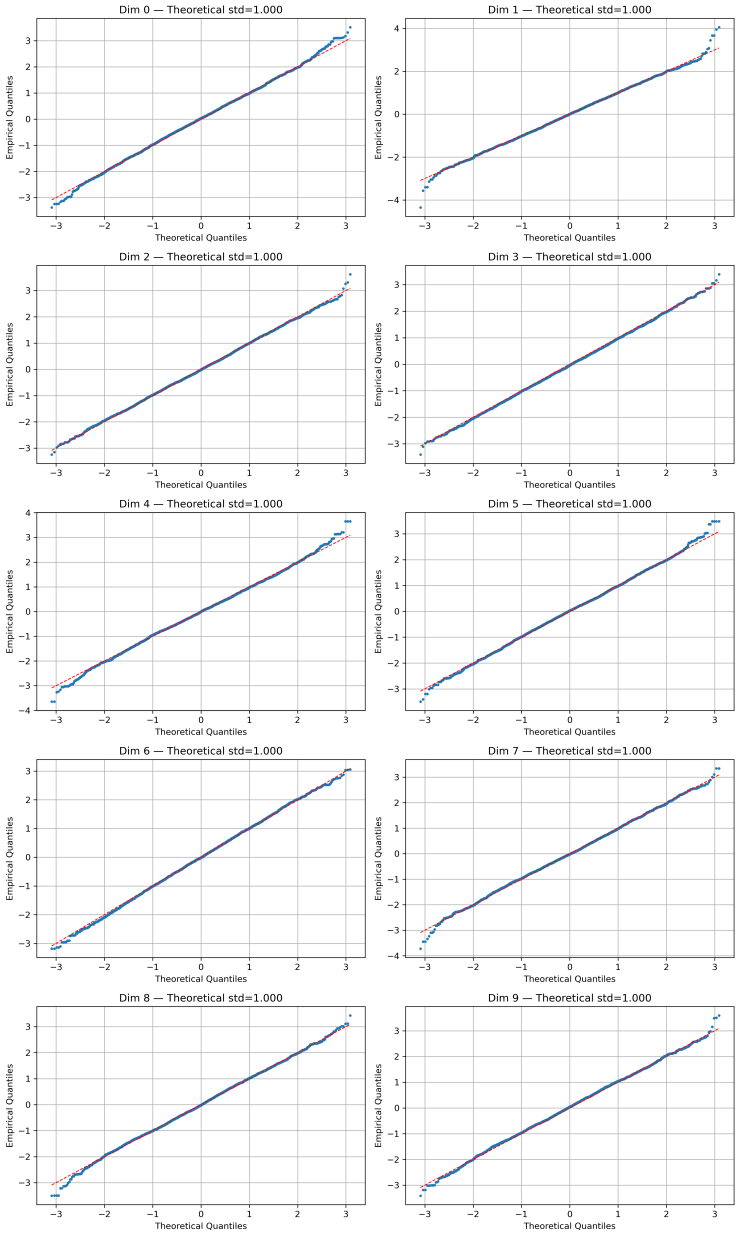
10-dim Gaussian distribution

10-dim Gaussian distribution



10-dim Gaussian distribution





Effective Sample Size (ESS) (1)

MCMC algorithms generate correlated samples $\theta_1, \theta_2, \ldots, \theta_M \sim p(\theta)$. Because these samples are dependent:

1000 MCMC samples $\neq$ 1000 independent samples

ESS estimates how many independent samples contain the same information as the M autocorrelated samples:

$$\text{ESS} = \frac{M}{1 + 2\sum_{s=1}^{M-1}\left(1 - \frac{s}{M}\right)\rho_s(f(\theta))}$$

- M: total number of generated samples
- ρ_s: autocorrelation of the chain at lag s
- $\left(1 - \frac{s}{M}\right)$: finite-sample weight factor

Effective Sample Size (ESS) (2)

ESS depends on the specific expectation $f(\theta)$ being estimated:

To evaluate samplers on high-dimensional targets (e.g., 250D Gaussian), the authors of the NUTS paper compute two separate ESS values for each dimension j:

- **First Moment (Mean):** $f(\theta) = \theta_j \implies \text{ESS}_j^{(\text{mean})}$
  - Measures efficiency in estimating the central location.
- **Second Central Moment (Variance):** $f(\theta) = (\theta_j - \mu_{\text{true}})^2 \implies \text{ESS}_j^{(\text{variance})}$
  - Measures efficiency in capturing uncertainty and scale.
  - Requires a long baseline run (e.g., 50,000 samples) to obtain μ_{true}.

Modern ESS diagnostics

Modern tools (PyMC / ArviZ) default to rank-normalized ESS (Vehtari et al., 2021):

- **Bulk ESS** (`"bulk"`): Rank-based measure of sampling efficiency in the center.
- **Tail ESS** (`"tail"`): Efficiency for extreme quantiles (5% and 95%).

Our Methodology: Despite modern defaults, in this project we use $\mathbf{ESS}^{(\mathbf{mean})}$ and $\mathbf{ESS}^{(\mathbf{variance})}$ to remain strictly coherent with the benchmark in the original NUTS paper (Hoffman & Gelman, 2014).

Aggregating ESS

Since ESS is computed per scalar component, we compute a single ESS separately for each dimension. We then aggregate:

1. **Joint Multi-Chain Calculation:** All parallel chains (e.g., 4 chains) are fed into the estimator simultaneously to penalize poor mixing, yielding a value of both $\mathrm{ESS}_j^{(\mathrm{mean})}$ and $\mathrm{ESS}_j^{(\mathrm{variance})}$ for each dimension
2. **Conservative Dimension Reduction:**
   - *Per dim. j:* Take the minimum of mean and variance ESS:

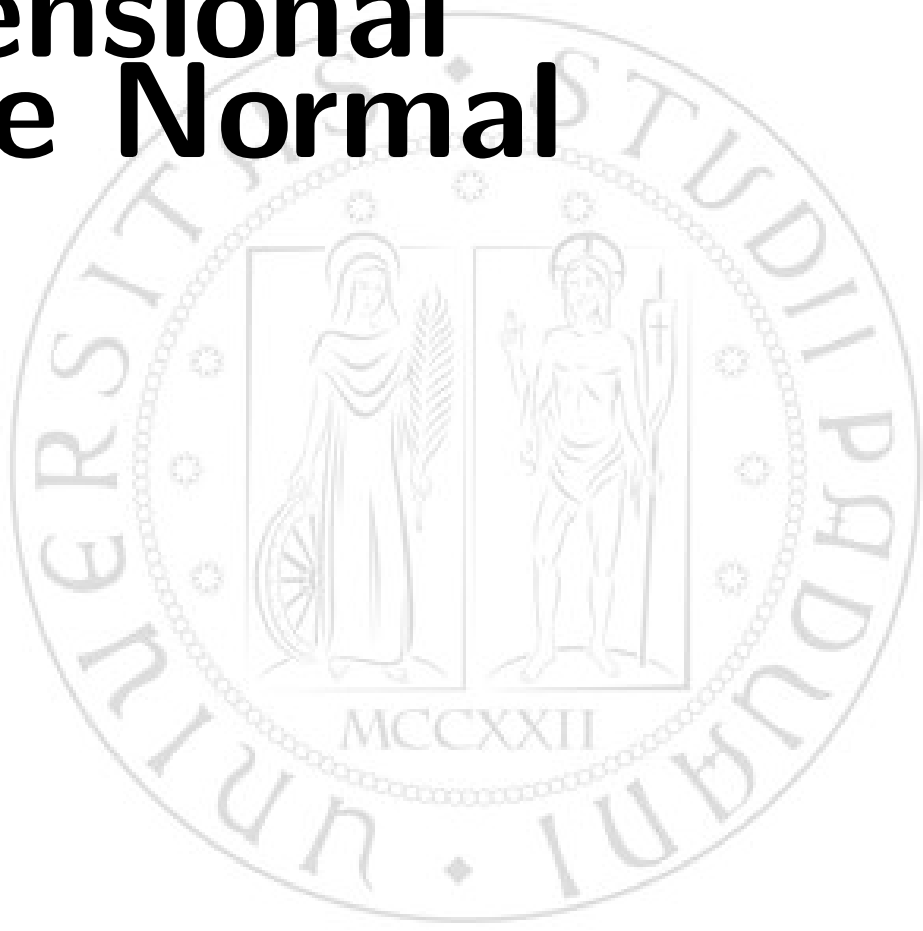
$$\mathrm{ESS}_j = \min\left(\mathrm{ESS}_j^{(\mathrm{mean})}, \mathrm{ESS}_j^{(\mathrm{variance})}\right)$$

   - *Across all dimensions:* Take the worst-case dimension:

$$\mathrm{ESS}_{\mathrm{final}} = \min_{j \in \{1,\ldots,250\}} \mathrm{ESS}_j$$

Result: A single, conservative metric reflecting the worst-explored feature of the distribution.

250-Dimensional Multivariate Normal

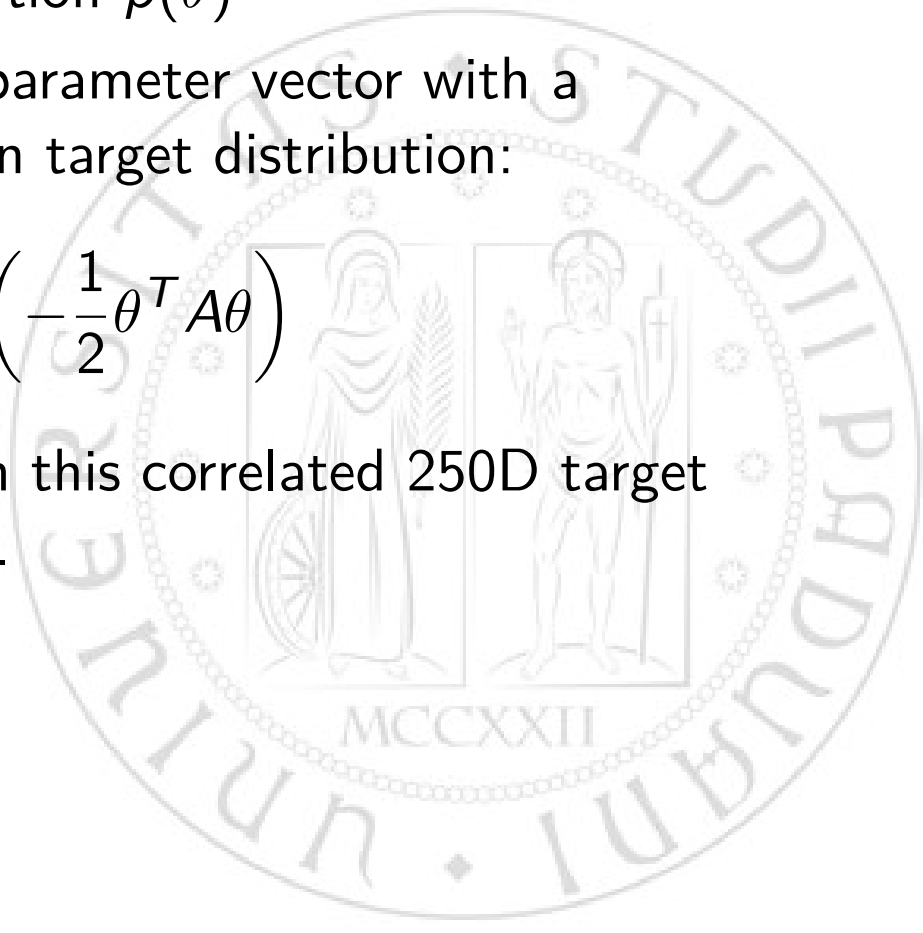


The Dataset & Target Distribution

1. This synthetic experiment directly samples from a high-dimensional target distribution $p(\theta)$
2. $\theta \in \mathbb{R}^{250}$ is a 250-dimensional parameter vector with a zero-mean multivariate Gaussian target distribution:

$$p(\theta) \propto \exp\left(-\frac{1}{2}\theta^T A\theta\right)$$

Our goal: sample efficiently from this correlated 250D target space.



Precision Matrix

The target precision matrix $A = \Sigma^{-1}$ dictates the geometry of the target space:

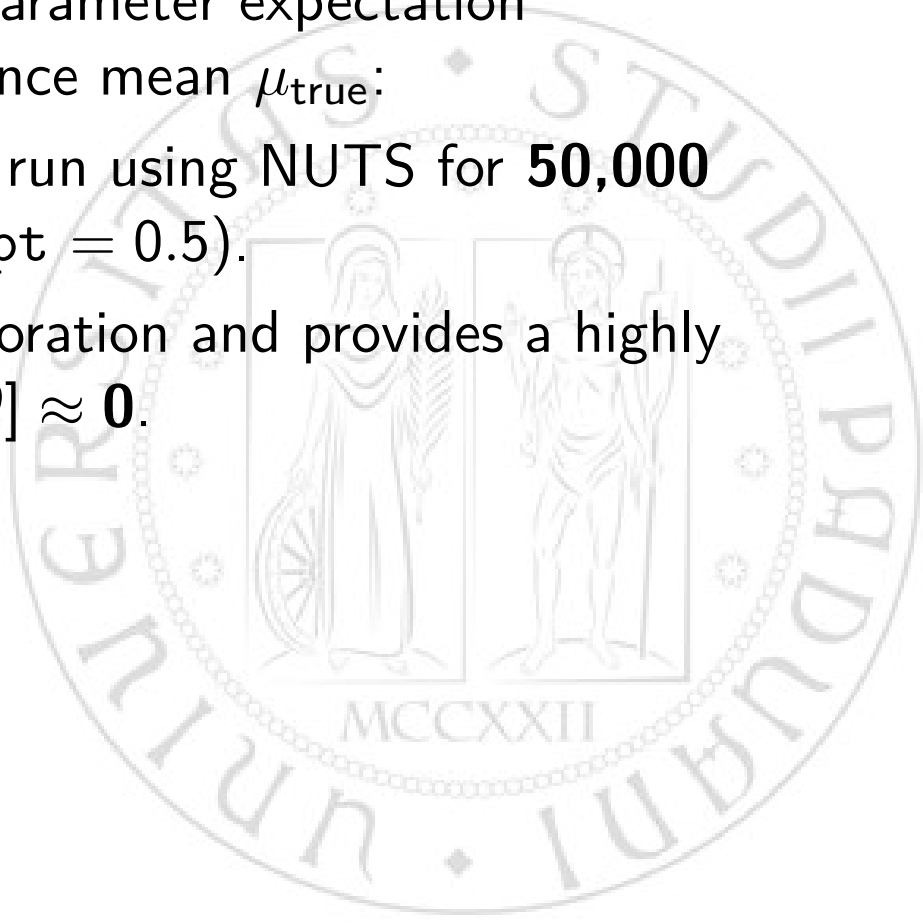
- A is generated randomly from a **Wishart distribution** $\mathcal{W}_D(I, \text{df} = 250)$ with identity scale matrix I and 250 degrees of freedom.
- **Role of Wishart:** Guaranteed to yield a symmetric, positive-definite matrix that creates strong correlations and complex, non-isotropic geometry.
- To ensure fair comparison across all runs and samplers, the exact same matrix A is fixed across all experiments.

High Correl. + High Dim. = Ideal Benchmark for HMC vs. NUTS

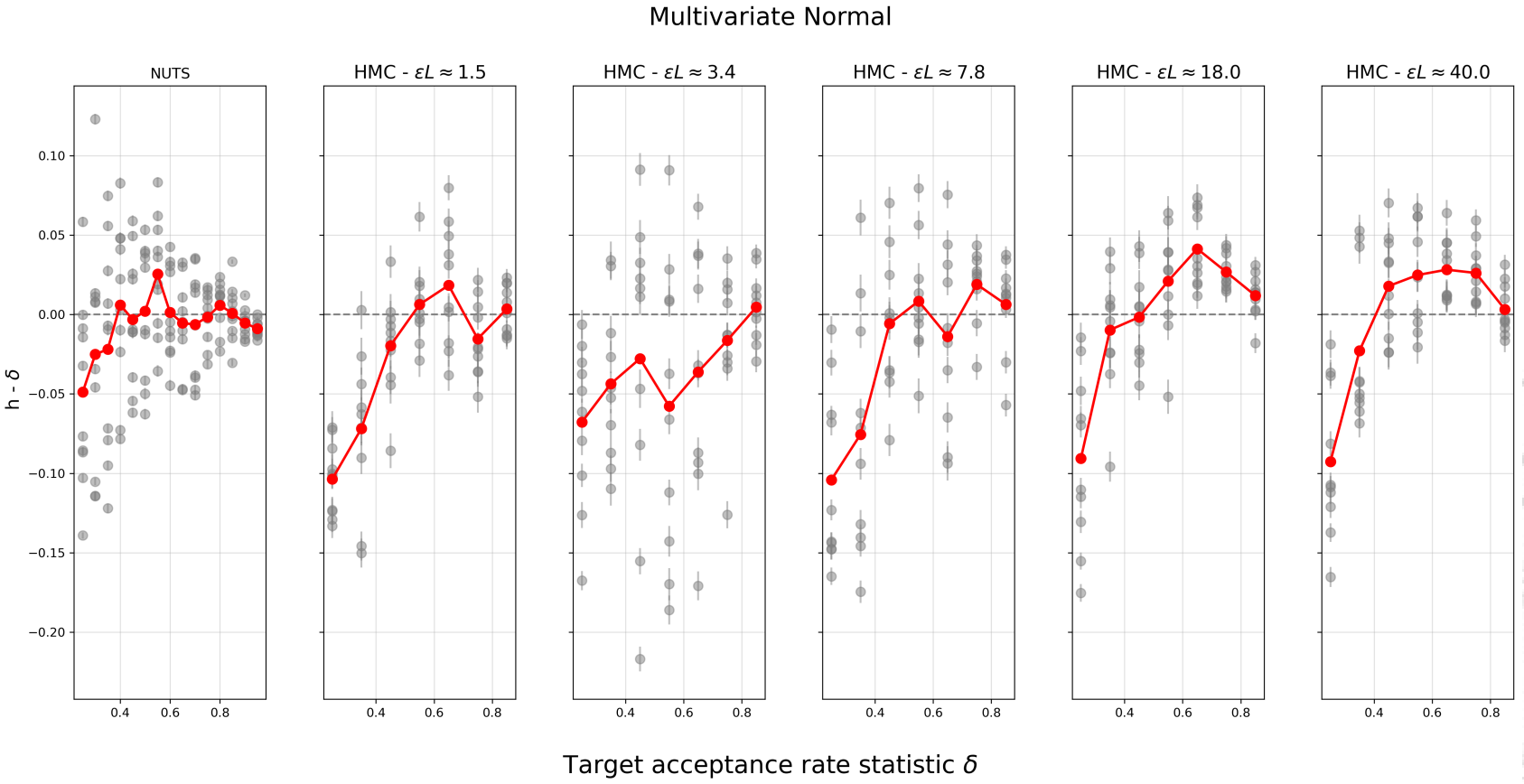
Baseline Parameter Estimation

To compute $\text{ESS}^{(\text{variance})}$, the true parameter expectation $f(\theta) = (\theta - \mu_{\text{true}})^2$ requires a reference mean μ_{true}:

- We executed an initial baseline run using NUTS for **50,000 iterations** (with `target_accept` $= 0.5$).
- The long run ensures deep exploration and provides a highly precise estimate for $\mu_{\text{true}} \approx E[\theta] \approx \mathbf{0}$.

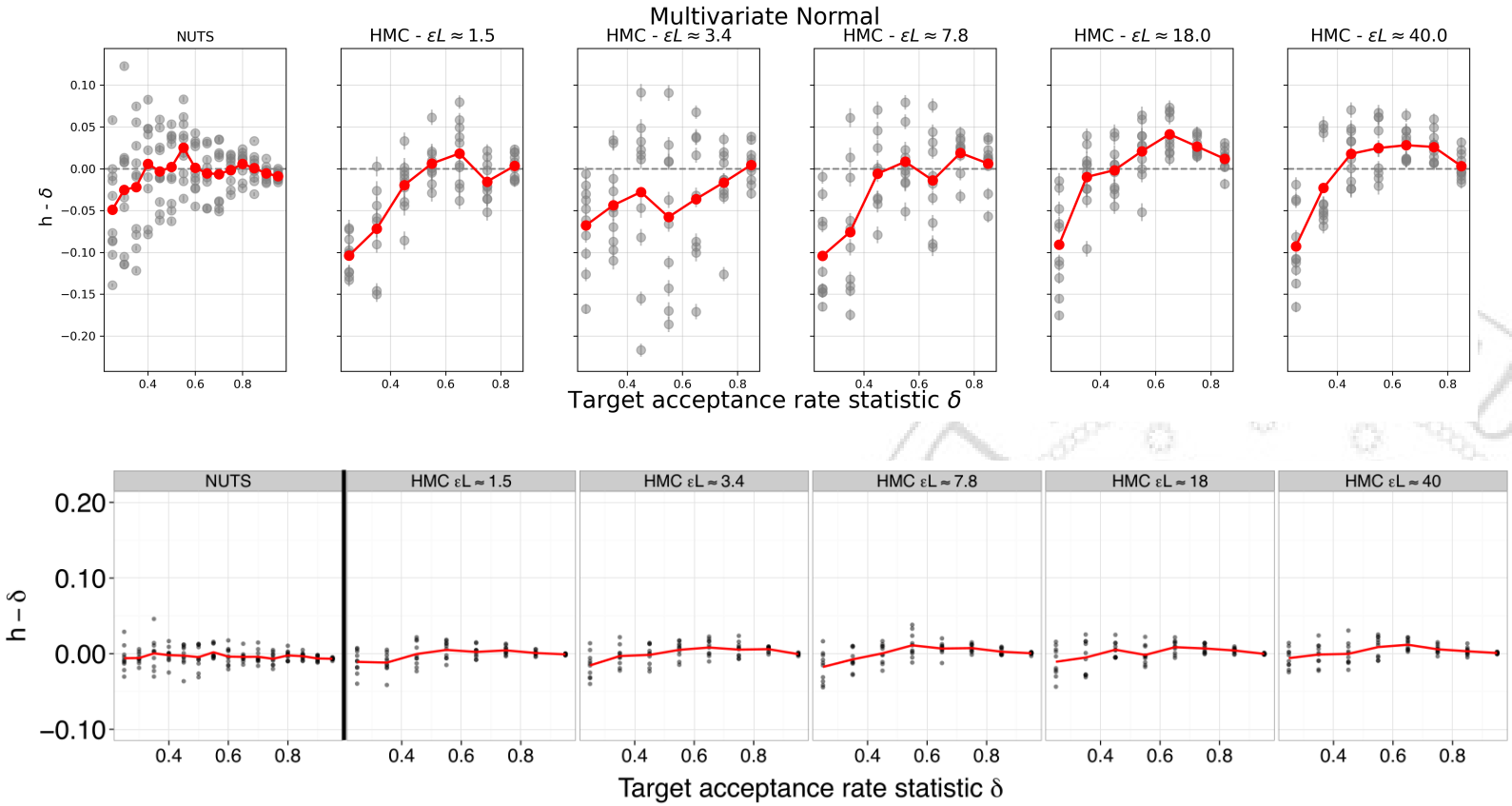


Target Acceptance (δ)



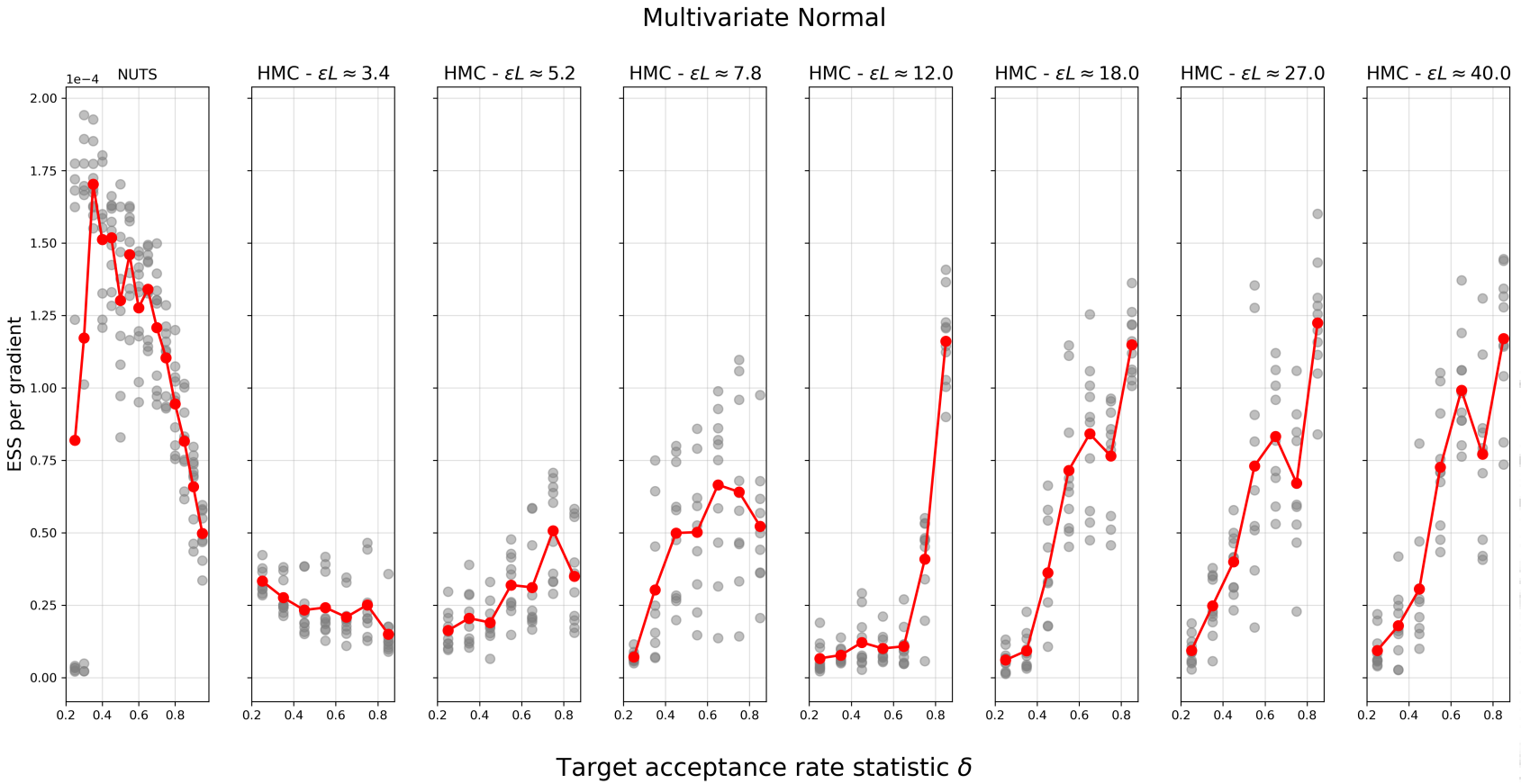
Each gray point is obtained by averaging across all 4 chains. The computation has been repeated 10 times for each value of δ. In red the average across all repetitions.

Target Delta (δ) comparison



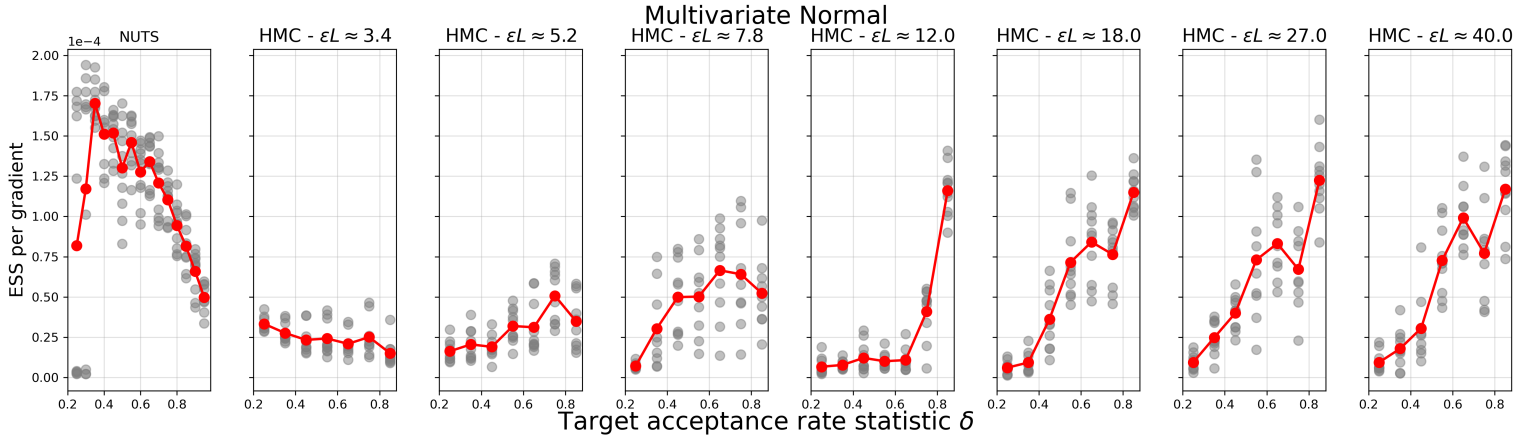
Above: our results for the 250D Gaussian. Below: the original paper results. Each point's distance from the x-axis shows how effectively the dual averaging algorithm tuned the step size ϵ for a single experiment.

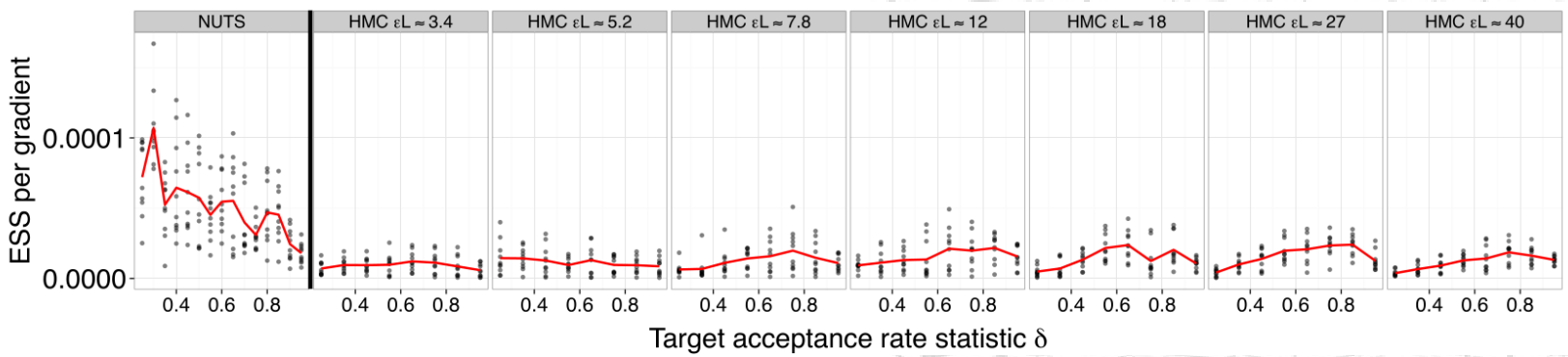
ESS per gradient



Each gray point is obtained by averaging across all 4 chains. The computation has been repeated 10 times for each value of δ. In red the average across all repetitions.

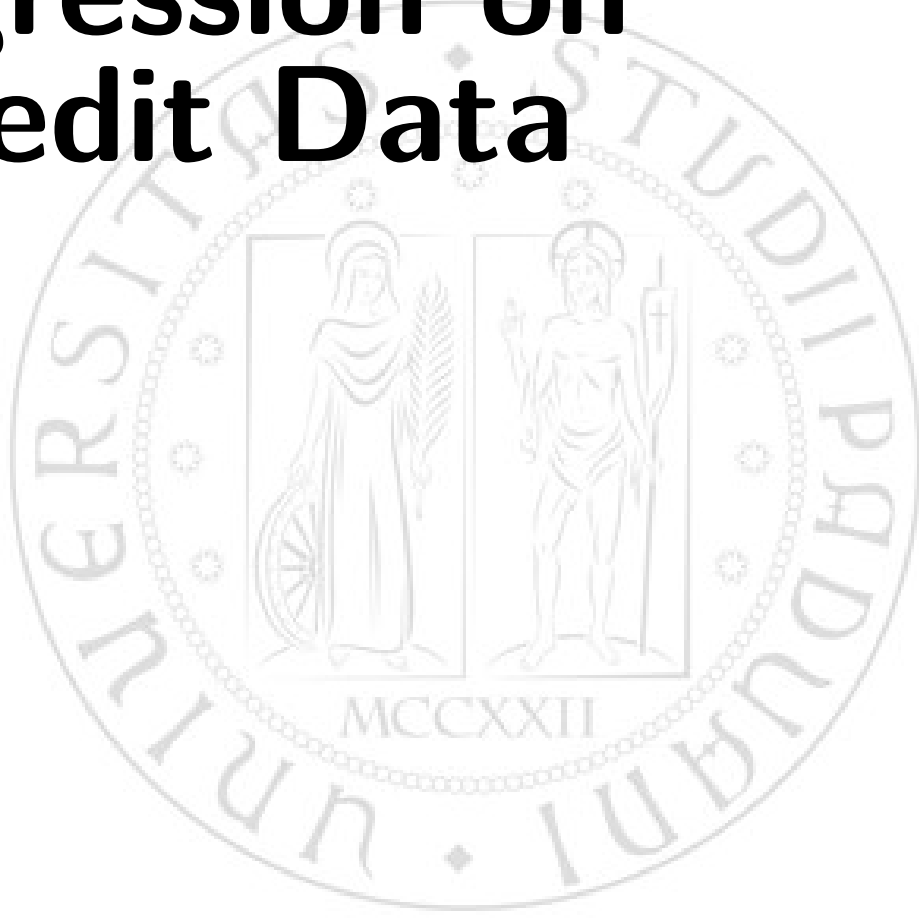
ESS per gradient comparison





Above: our results for the 250D Gaussian. Below: the original paper results.

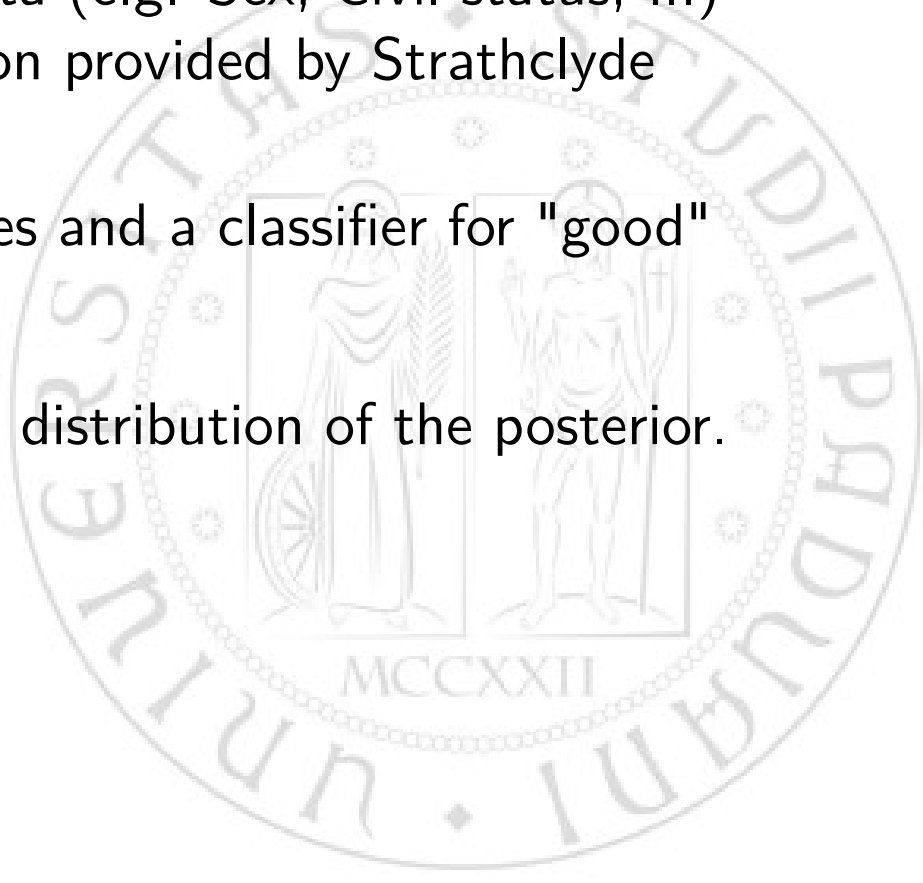
Logistic Regression on German Credit Data



The Dataset

1. Original dataset contains both numerical (e.g. Age, credit amount, ...) and categorical data (e.g. Sex, Civil status, ...) $\rightarrow$ we used the numerical version provided by Strathclyde University.
2. 1000 individuals with 24 features and a classifier for "good" (+1) and "bad" (-1) creditors.

Our goal: sample from the target distribution of the posterior.



The Model

Bayesian Logistic Regression model.

The target distribution is $p(\alpha, \beta \mid X, y) \propto p(y|X, \alpha, \beta)\, p(\alpha)\, p(\beta)$

$$\propto \exp\left(-\sum_i \log(1 + \exp\{-y_i\,(\alpha + x_i \cdot \beta)\}) - \frac{\alpha^2}{2\sigma^2} - \frac{1}{2\sigma^2}\,\beta \cdot \beta\right)$$

where:

- x_i is a 24-dimensional vector of numerical predictors associated with a customer i.
- y_i is $+1$ if the customer should receive credit, -1 otherwise.
- α is a scalar intercept term.
- β is a vector of 24 regression coefficients.
- All predictors are normalized to have zero mean and unit variance.
- α and each element of β are given given weak zero-mean normal priors with a **fixed** variance $\sigma^2 = 100$.

The Model: Likelihood

We assume that the probability of a customer being "good" depends on a linear combination of the features:

$$\eta_i = \alpha + x_i\beta$$
$$p(y_i = 1 \mid x_i, \alpha, \beta) = \sigma(\eta_i)$$

where $\sigma(\eta_i) = \frac{1}{1+e^{-\eta_i}}$ is the sigmoid logistic function.
Therefore, because $y = \{-1, +1\}$

$$\begin{cases} p(y_i = 1|x_i) = \sigma(\eta_i) \\ p(y_i = -1|x_i) = 1 - \sigma(\eta_i) = \frac{e^{-\eta_i}}{1+e^{-\eta_i}} = \frac{1}{1+e^{+\eta_i}} \end{cases}$$

And we obtain the compact form:

$$p(y_i|x_i) = \sigma(y_i\eta_i) = \frac{1}{1 + e^{-y_i\eta_i}}$$

Realized average acceptance probability h

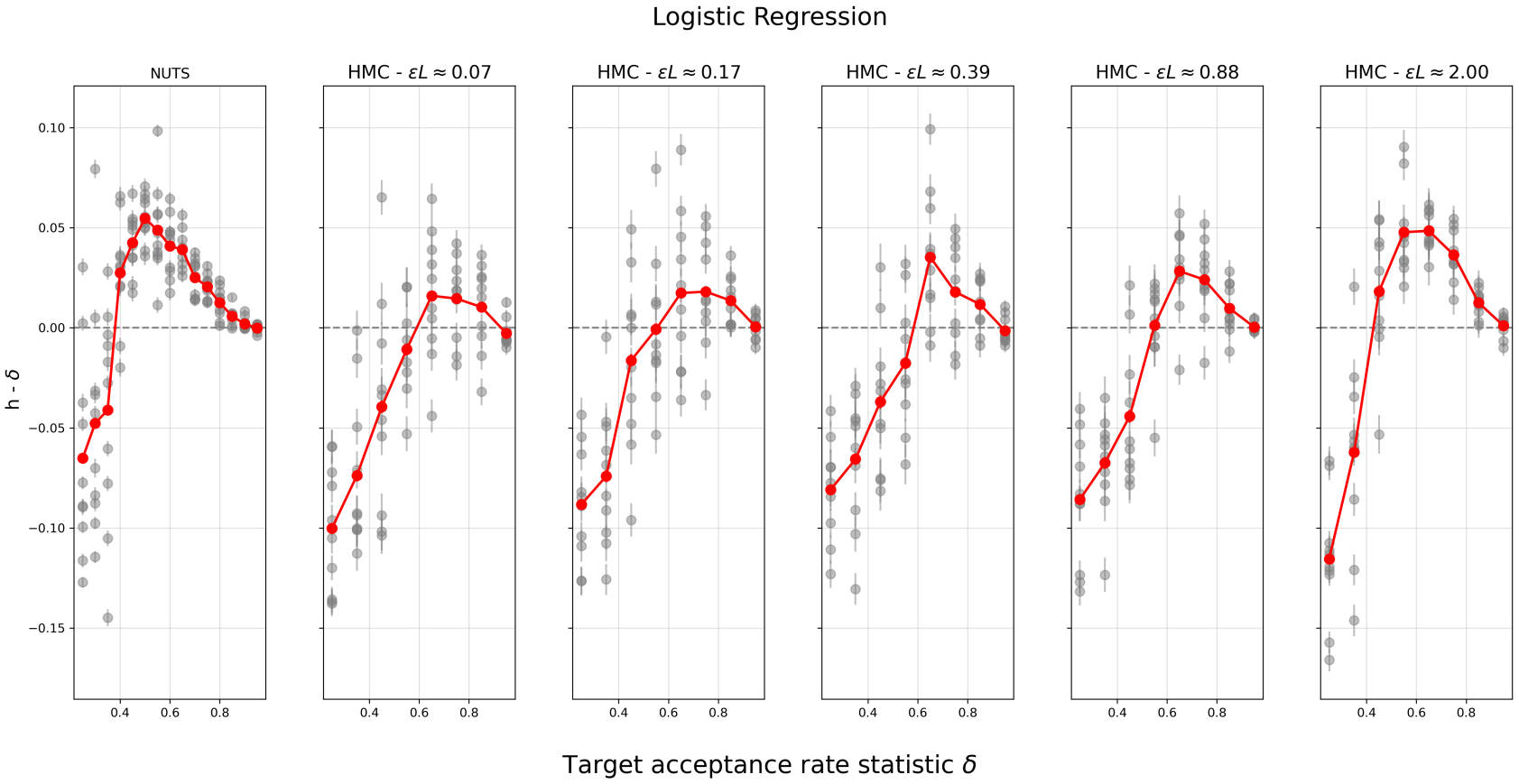
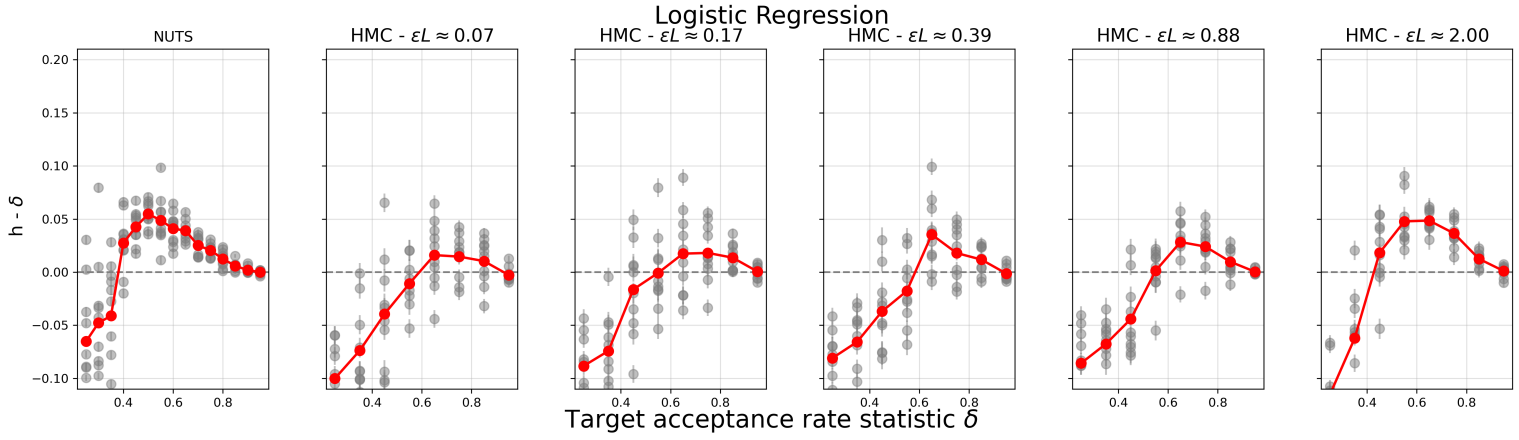
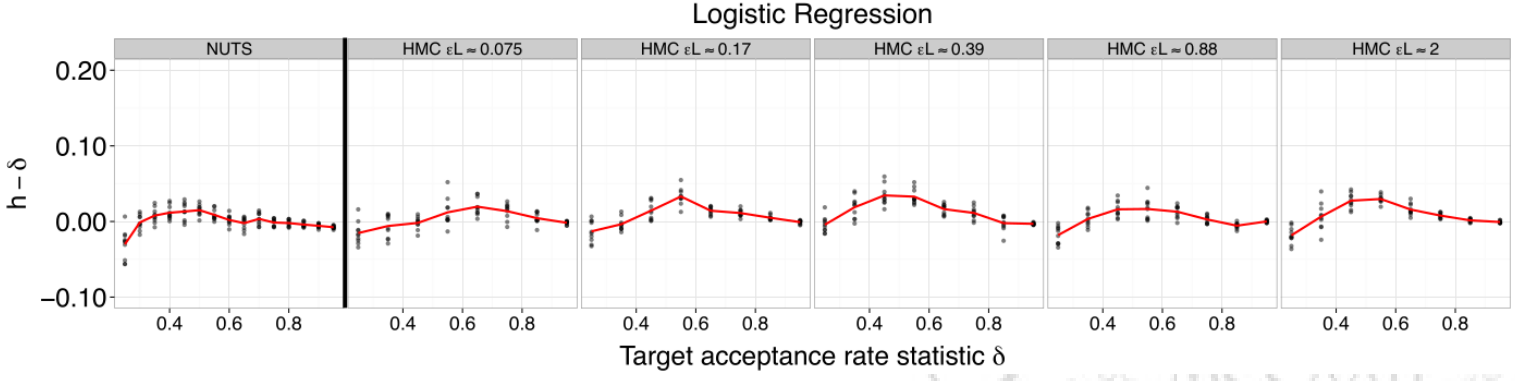


Figure: Each gray point is obtained by averaging across all 4 chains. The computation has been repeated 10 times for each value of δ. In red the average across all repetitions.

Acceptance probability comparison with paper



(a) Our results



(b) Paper's results.

ESS efficiency

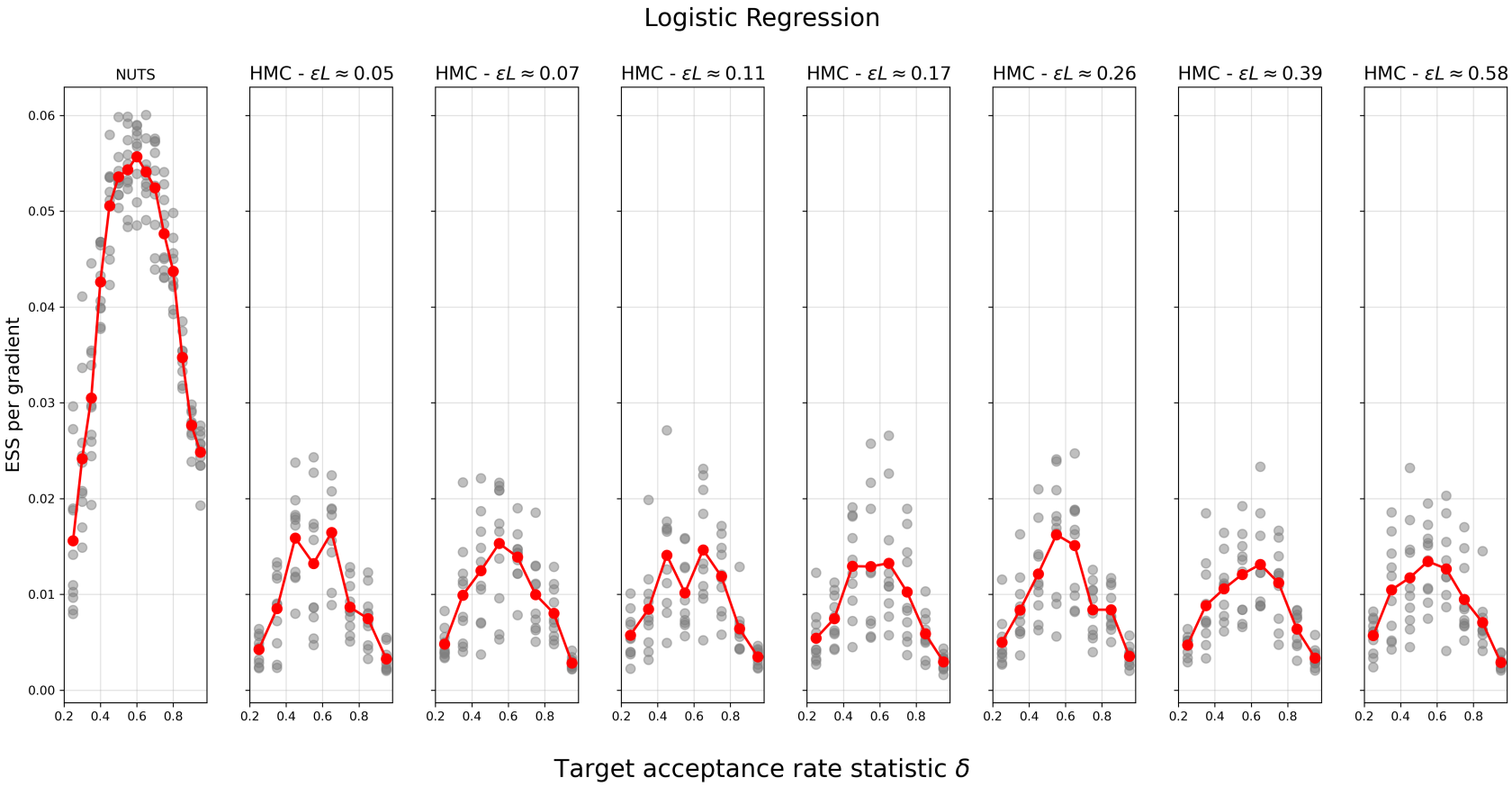
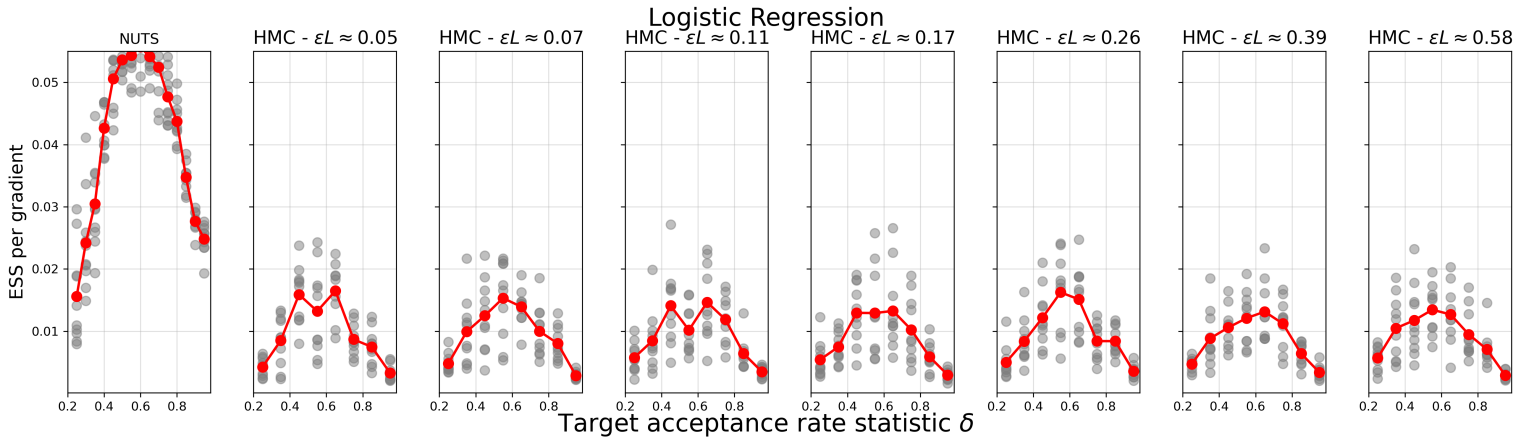
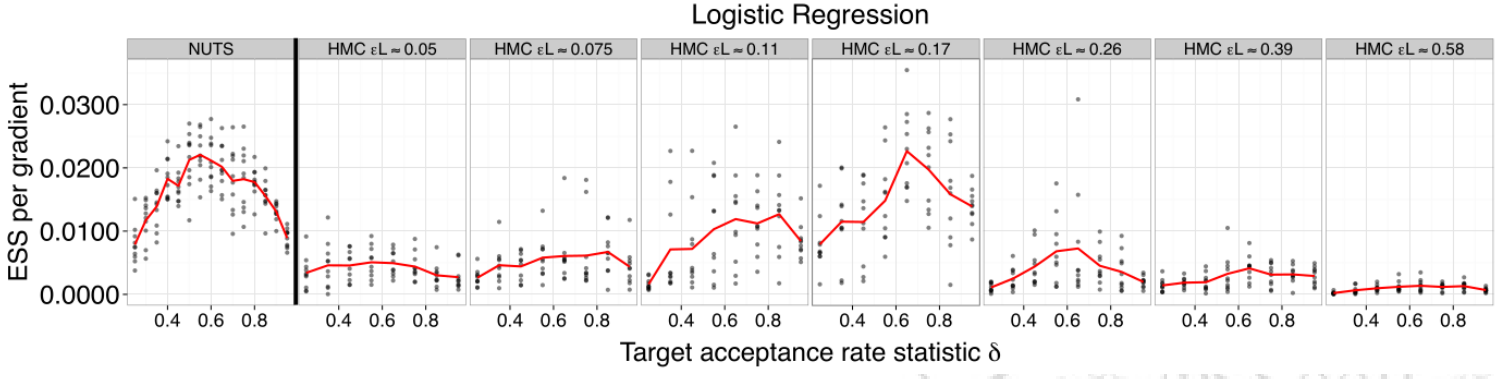


Figure: Each gray point is obtained by taking the minimum ESS per gradient across all 4 chains for all variables, considering both mean and variance. In red the average across all repetitions.

ESS efficiency comparison with paper

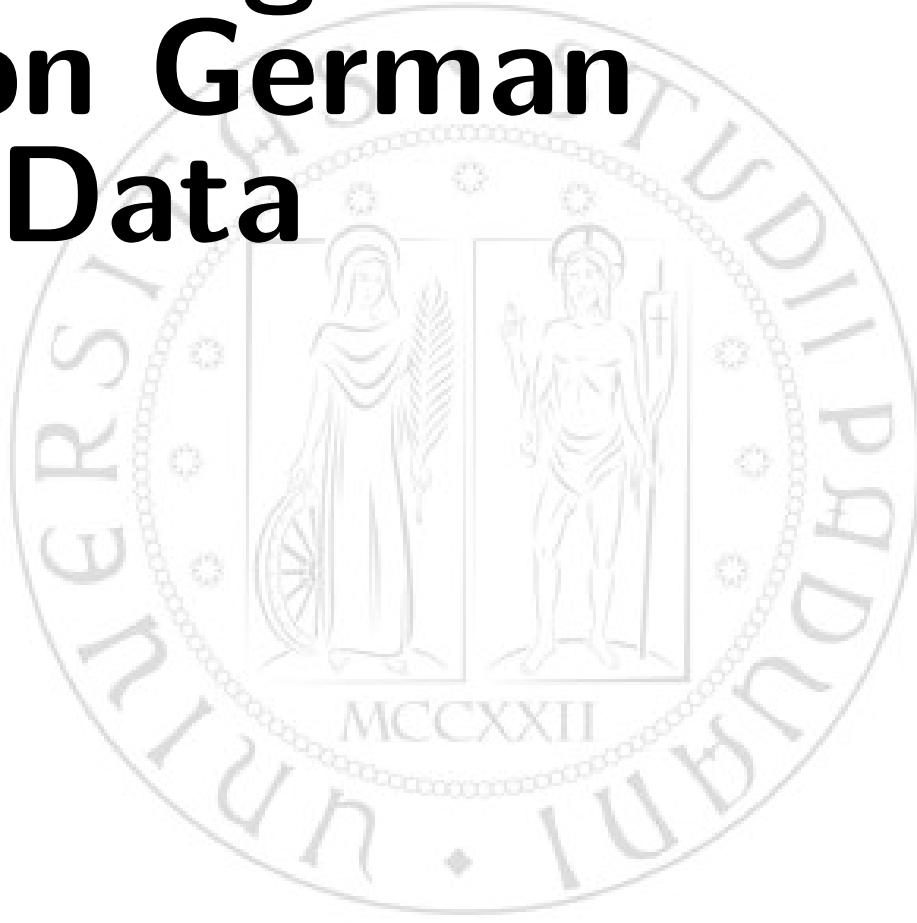


(a) Our results



(b) Paper's results.

Hierarchical Logistic Regression on German Credit Data



Changes from previous model

1. We expand the predictor vectors by including two-way interactions ($x_i \cdot x_j$, $i \neq j$), resulting in $\binom{24}{2} = 276 + 24 =$ 300-dimensional vectors of predictors $\mathbf{x}$ and a 300-dimensional vector of coefficients β.
2. The variance parameter in the prior on α and β is given an exponential prior and estimated as well.
3. We set $\lambda = 0.01$, yielding a weak exponential prior distribution on σ^2, whose mean and standard deviation are 100.

$$\Downarrow$$

These changes make for a more challenging problem: the posterior is in higher dimensions and the variance term σ^2 interacts strongly with the remaining 301 variables.

The paper's posterior

$$p(\alpha, \beta, \sigma^2 \mid X, y) \propto p(y|X, \alpha, \beta)\, p(\beta|\sigma^2) p(\alpha|\sigma^2) p(\sigma^2) \propto$$

$$\exp\left(-\sum_i \log(1 + \exp\{-y_i x_i \cdot \beta\}) - \frac{\alpha^2}{2\sigma^2} - \frac{1}{2\sigma^2}\beta \cdot \beta - \frac{N}{2}\log\sigma^2 - \lambda\sigma^2 \right)$$

where N $= 1000$ is the number of customers.

However...

that should instead be $\frac{\mathbf{D+1}}{\mathbf{2}}$ where D is the dimension of β (300).

(Also the intercept term α is missing in the (log)-likelihood)

Motivation

Now the variance σ^2 on α and β is inferred from data, therefore in the log-posterior for these parameters we get:

$$\log p(\alpha \mid \sigma^2) = -\frac{1}{2}\log\left(2\pi\sigma^2\right) - \frac{\alpha^2}{2\sigma^2}$$

$$\log p(\beta \mid \sigma^2) = -\frac{D}{2}\log\left(2\pi\sigma^2\right) - \frac{\beta^T\beta}{2\sigma^2}$$

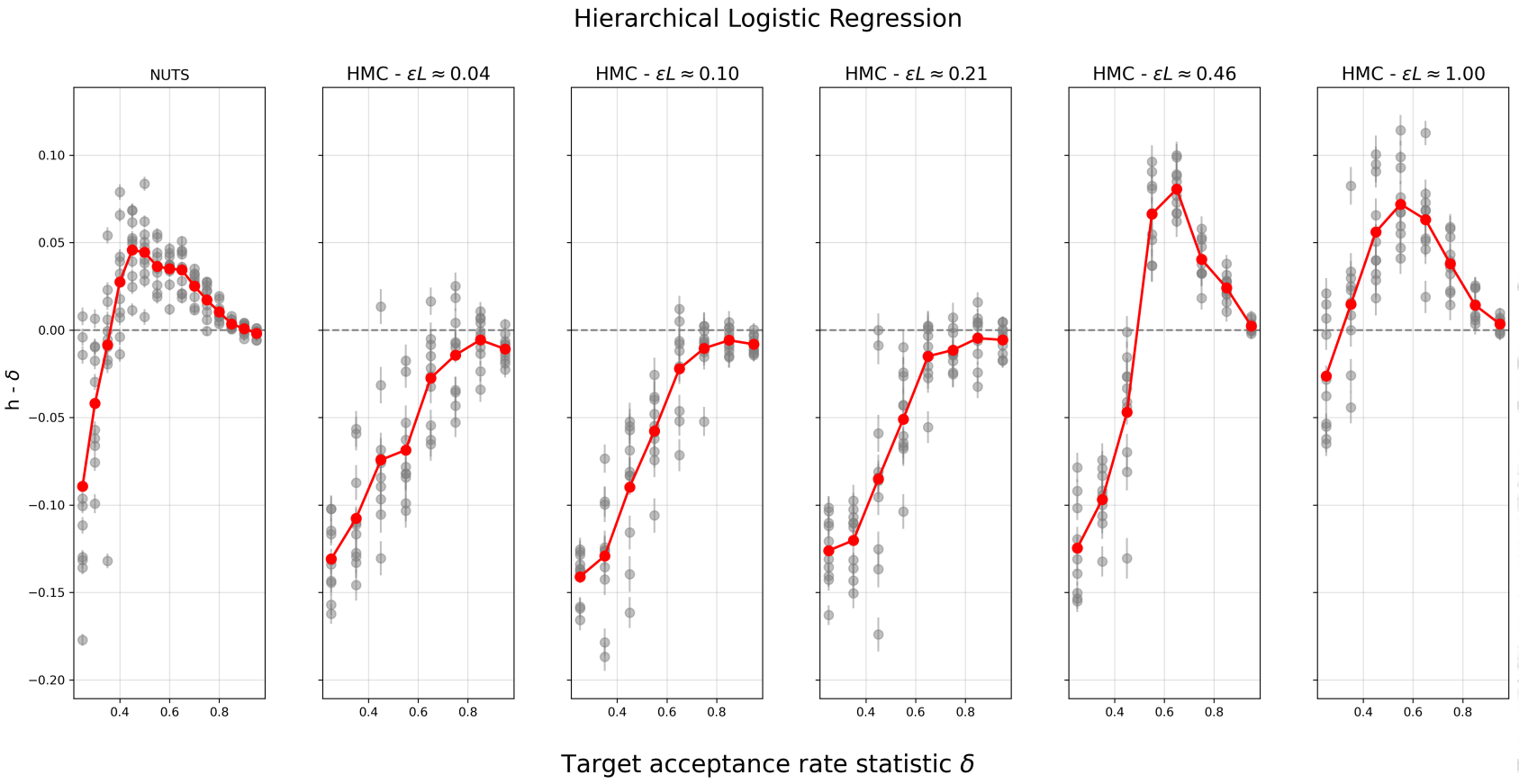
So their total contribution to the log-posterior is:

$$-\frac{\alpha^2 + \beta^T \cdot \beta}{2\sigma^2} - \frac{D+1}{2}\log(\sigma^2)$$

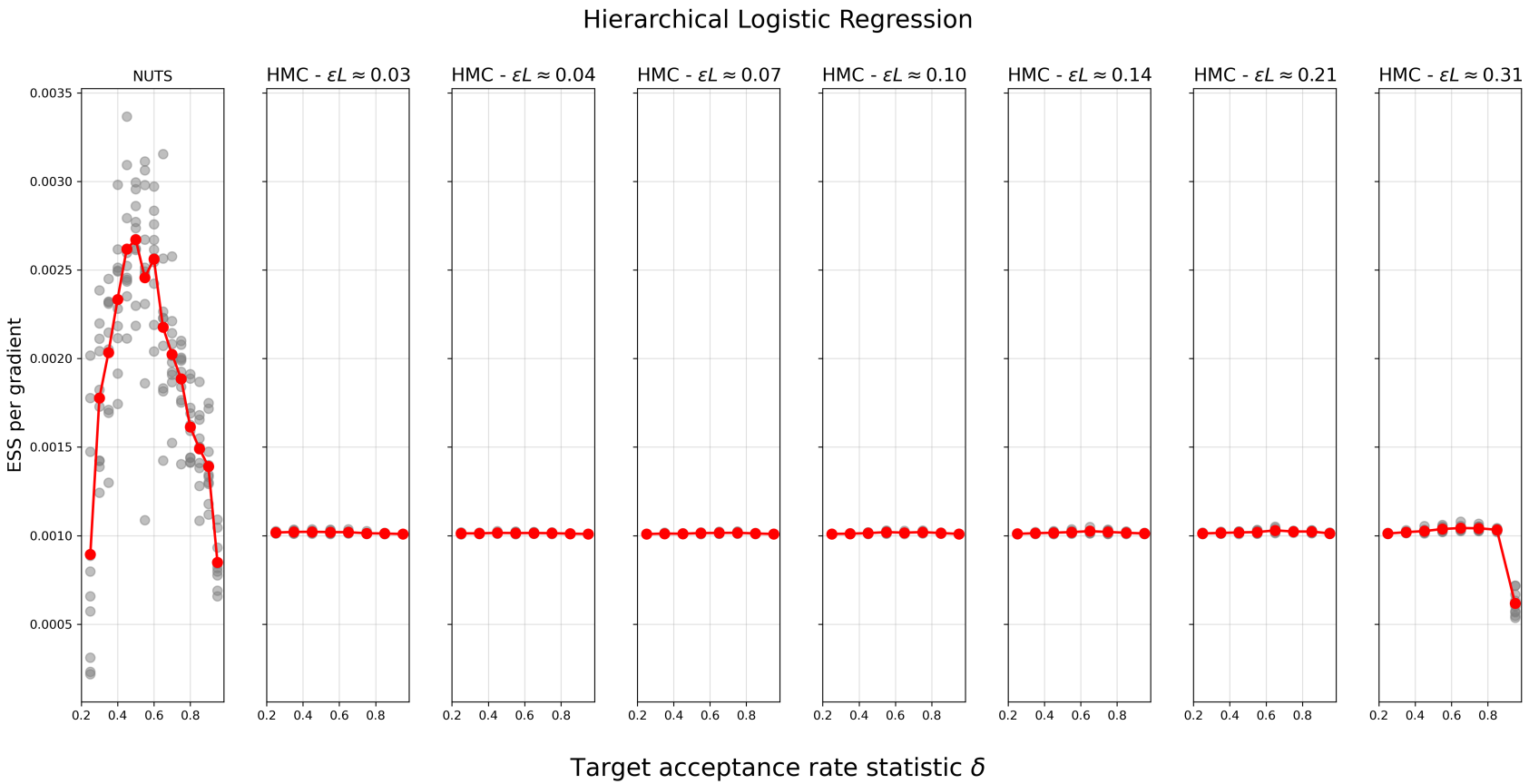
Therefore, the log-posterior we sampled from is proportional to:

$$-\sum_i \log(1+\exp\{-y_i\left(\alpha + x_i \cdot \beta\right)\}) - \frac{\alpha^2}{2\sigma^2} - \frac{\beta^T\beta}{2\sigma^2} - \frac{D+1}{2}\log\sigma^2 - \lambda\sigma^2$$

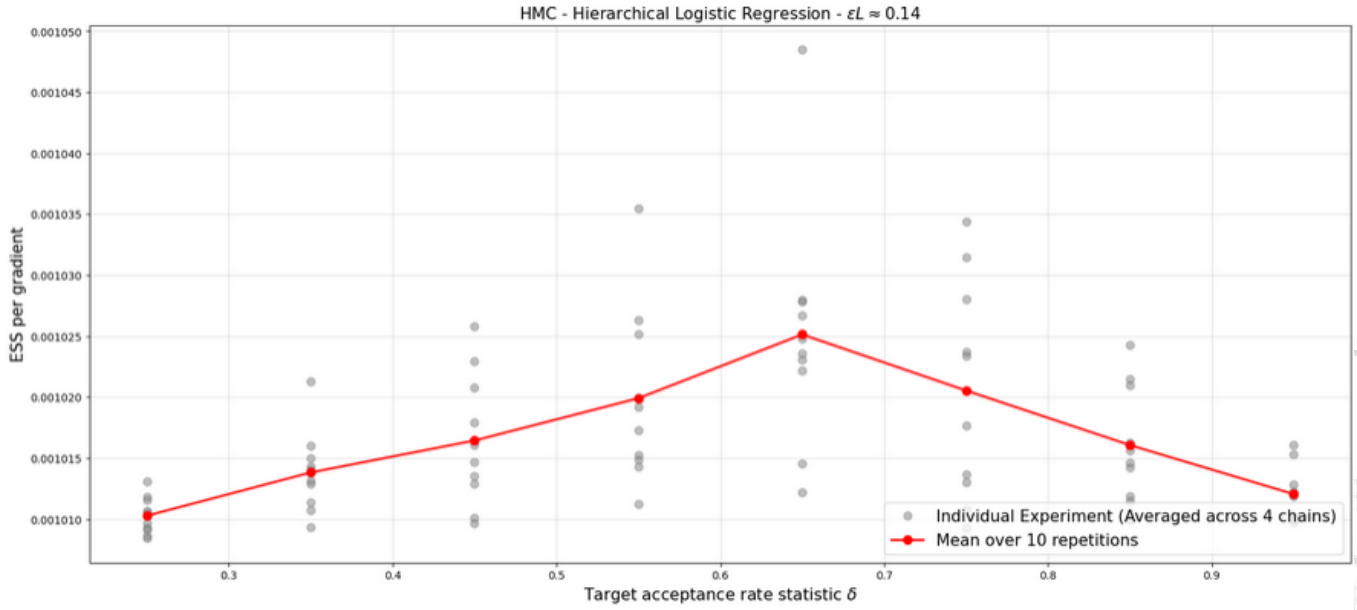
Realized average acceptance probability h

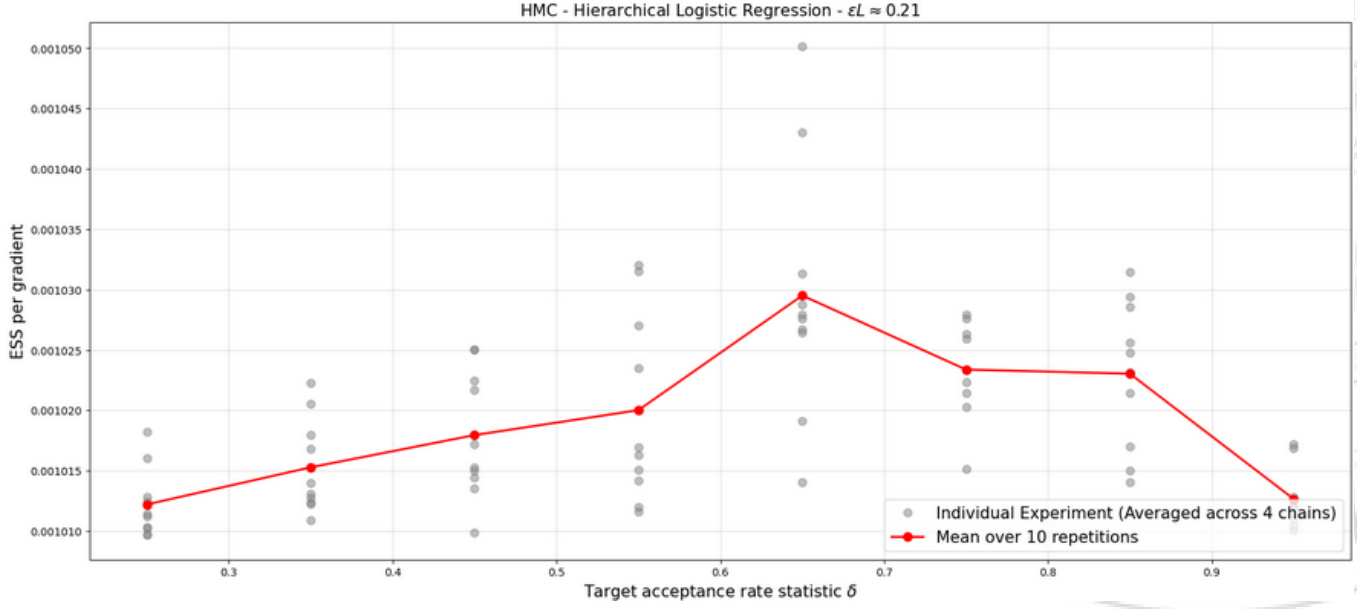


ESS efficiency

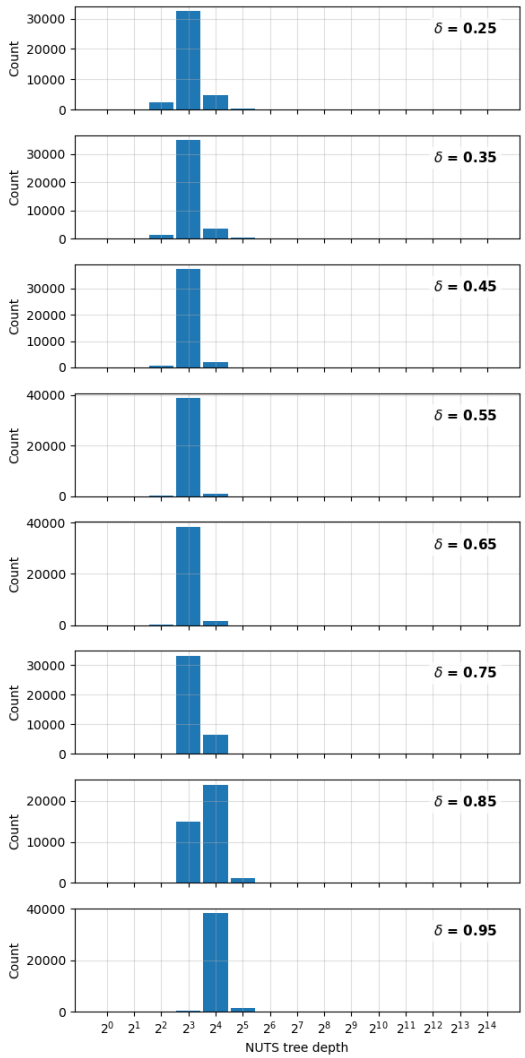


ESS efficiency: HMC example

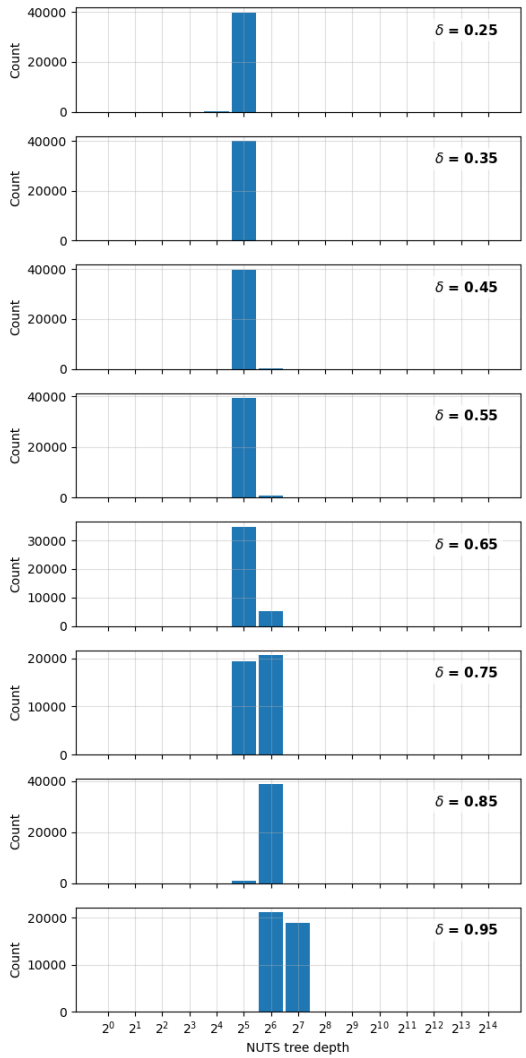




NUTS Tree Depth Histogram



(a) Logistic Regression Tree Depths.



(b) Hierarchical LR Tree Depths.